A Space-Efficient Distance Labeling for Dynamic Weighted Small-World Networks

Ruizhong Wu, Xinjie Zhou
DSA Thrust, HKUST(GZ)
{rwu601,xzhouby}@connect.hkust-gz.edu.cn

Mengxuan Zhang
The Australian National University
mengxuan.zhang@anu.edu.au

Lei Li
DSA & INTR Thrust, HKUST(GZ)
Department of CSE, HKUST
thorli@ust.hk

ABSTRACT

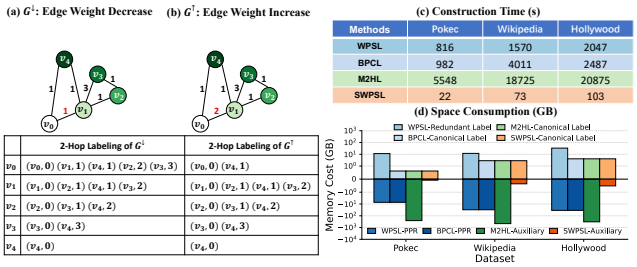
Answering the shortest distance query is an important graph analytic task, and 2-hop labeling is the state-of-the-art index that returns the distance in microseconds. However, it can hardly work on weighted small-world networks in dynamic environments, because the existing solutions suffer from large redundant labels during construction and large auxiliary structures during maintenance, both of which can be ten times larger than the already large index. In this paper, we propose a space-efficient 2-hop labeling SWPSL that eliminates both of them. Our redundancy analysis motivates a hybrid construction that first runs PLL from the highest-ranked hubs. Their canonical labels prune redundant candidates during parallel propagation, followed by a final cleanup of the remaining redundancy. After that, we propose a *remove-and-recover* paradigm that maintains the labels through pruned propagation without the large pruning point record, and removes the newly introduced redundancy with restricted checking. Experiments show that SWPSL is not only the most space-efficient dynamic 2-hop labeling, but its construction is also up to 100X faster than the existing methods, and it handles a thousand updates in seconds while keeping only one byte per label for maintenance.

PVLDB Reference Format:
Ruizhong Wu, Xinjie Zhou, Mengxuan Zhang, and Lei Li. A Space-Efficient Distance Labeling for Dynamic Weighted Small-World Networks . PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/CSU-WRZ/SWPSL.

1 INTRODUCTION

Many real-world networks share the "small-world effect" [59]: most vertices are highly clustered and reachable from each other within *a small number of steps*. Beyond topology, these edges also carry semantic weights [11]: 1) In social networks, the edge weights reflect the proximity or trust between users derived from their interaction frequency and similarity [9, 18]. For instance, the LDBC-SNB [22] weights friendship edge by $\max(\text{round}(40-\sqrt{numInteractions}), 1)$ and asks for the *cheapest interaction path* between users (BI19). 2)



	2-Hop Labeling of G^1	2-Hop Labeling of $G^\uparrow$
v_0	$(v_0,0)$ $(v_1,1)$ $(v_4,1)$ $(v_2,2)$ $(v_3,3)$	$(v_0,0)$ $(v_4,1)$
v_1	$(v_1,0)$ $(v_2,1)$ $(v_4,1)$ $(v_3,2)$	$(v_1,0)$ $(v_2,1)$ $(v_4,1)$ $(v_3,2)$
v_2	$(v_2,0)$ $(v_3,1)$ $(v_4,2)$	$(v_2,0)$ $(v_3,1)$ $(v_4,2)$
v_3	$(v_3,0)$ $(v_4,3)$	$(v_3,0)$ $(v_4,3)$
v_4	$(v_4,0)$	$(v_4,0)$

Methods	Pokec	Wikipedia	Hollywood
WPSL	816	1570	2047
BPCL	982	4011	2487
M2HL	5548	18725	20875
SWPSL	22	73	103

Figure 1: 2-Hop Labeling of a Dynamic Graph and Its Space and Time Costs (SWPSL in Orange)

In communication networks [51], the weights are transmission latencies or bandwidth costs, so the optimal routing strategy requires the shortest weighted path. 3) In location-based social networks [16, 63, 64], the weights are the spatial distances between users, and a shorter weighted distance implies a closer relationship. 4) In protein interaction networks [13], the weights quantify the interaction confidence and functional similarity [31, 50], and the shortest weighted path from a disease gene to a drug target reveals the therapeutic pathway through druggable intermediates [49]. 5) In web graphs [12], the weights are the negative log probabilities of user clicks [60] or the semantic dissimilarities between pages [48], so the shortest path is the most probable navigation route or the most relevant conceptual bridge between two topics. Therefore, computing the shortest distance on weighted small-world networks is a fundamental building block of graph analytics.

The shortest distance can be answered in μs with *2-hop labeling* [7, 15, 17, 32, 38, 43, 52], which pre-computes a label set $L(v) = \{(u, dist(v,u))\}$ for each vertex v to store its distances to the hubs u. Then, the distance from s to t is the minimum of $dist(s,u)+dist(u,t)$ over the common hubs u of $L(s)$ and $L(t)$. For example, the common hub of $L(v_0)$ and $L(v_3)$ in Figure 1(b) is v_4, so $dist(v_0,v_3) = 1+3 = 4$. A good 2-hop labeling should satisfy two essential properties: **2-hop cover**, which guarantees that the distance between any two vertices can be derived from their labels, and **minimality**, which guarantees that no redundant label exists. When the network changes as in Figure 1(a) and (b), the labels have to be maintained to preserve the *2-hop cover* for correctness, while the *minimality* has to be kept during both construction and maintenance for efficiency.

These two properties have been achieved efficiently on dynamic road networks [67, 70, 71, 76] and dynamic unweighted small-world networks [65], but not on dynamic *weighted* small-world networks, due to the following two characteristics.

1) *Huge hub degrees*: a small-world network contains hub vertices with hundreds of thousands or even millions of neighbors. The tree decomposition-based labels [67, 71, 76] eliminate the vertices one by one and connect the neighbors of each eliminated vertex

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

with shortcuts, which suits road networks with small treewidth. However, eliminating a hub with d neighbors creates up to $d^2/2$ shortcuts, and the dense core of a small-world network has a large treewidth [44], so even the *Core-Tree* index for unweighted small-world networks has to stop the decomposition before the core and label the core separately. Accordingly, none of these methods has reported results on small-world networks, and only the search-based *PLL* [7] and the propagation-based *PSL* [33, 43, 65] can construct the labels on small-world networks.

2) *Short topological paths but a wide weighted distance range*: the small-world property concerns the unweighted topology. It does not guarantee that minimum-weight paths contain few edges. On unweighted graphs, this property enables the bit-parallel BFS of *PLL* and lets the distance-based propagation of *PSL*, which generates the labels of distance $d+1$ from those of distance d, converge within a few iterations. However, weighted edges force *PLL* to run the much slower *Dijkstra's*, while the distance-based propagation advances the distance by only one unit per iteration, so its iteration number is bounded by the numeric range of the weighted distances rather than by the small hop diameter. For example, on the *Yelp* network with real spatial weights, the distance-based *M2HL* [65] builds the index in 0.07s when the weights are rounded to the unit of 10^4km (*i.e.,* all the weights become 1), but it takes 1530s at the unit of 10m and runs out of memory at 1m. It should be noted that this characteristic is not a corner case of specific weight distributions: any real-valued weight has a wide numeric range after being discretized, and even the construction of the unweighted *Core-Tree* index [44] has to handle a weighted core graph with high degrees after the periphery is contracted. Therefore, the step-based *WPSL* [68, 69, 75] propagates labels one edge per round rather than by distance units. Its actual propagation depth depends on the weighted paths explored. However, it generates a large number of redundant labels (Figure 1(d), several times larger than the canonical labels), which prolong construction, query, and maintenance. What is worse, its maintenance relies on the *Pruning Point Record (PPR)*, which is several times larger than the labels.

Therefore, the space-consuming **redundant labels** and **auxiliary maintenance structures** prohibit 2-hop labeling from working on dynamic weighted small-world networks. Consequently, the path-related analytics are restricted to the simplified paradigm of finding the unweighted shortest paths first and then assigning weights to them [22], which degrades the analytic quality. To bridge this gap, we aim to design a space-efficient 2-hop labeling *SWPSL* for dynamic weighted small-world networks by eliminating both the redundant labels and the heavy auxiliary maintenance structures. However, it is non-trivial due to the following two challenges:

Challenge 1: *Redundant labels are inevitable in parallel propagation, so time efficiency and space efficiency are hard to achieve together.* A 2-hop labeling without redundancy has to be *canonical*: each vertex v has a unique order $r(v)$, and every hub u in $L(v)$ must have the highest order among all the vertices on the shortest paths between u and v. *PLL* [7] obtains the canonical labels by running pruned *Dijkstra's* strictly in the decreasing vertex order, but it is sequential and slow. *BPCL* [75] runs a batch of *Dijkstra's* in parallel and removes the redundancy within each batch, while *PSL* [33, 43, 65] and *WPSL* [68] propagate the labels in parallel in a vertex-centric fashion. Because the labels from the higher-order hubs may arrive several steps later than those from the lower-order hubs in *WPSL*, a label cannot be certified as canonical when it is generated. Consequently, the existing solutions remove the redundancy only after the whole construction, by which time the intermediate redundancy has already slowed down every step of the construction, whereas removing the redundancy at every step checks the same labels repeatedly. We first identify when a hub can certify the redundancy for good, which yields a direct one-pass removal that uses each hub as a witness only once. However, this removal still prunes late. The strongest witnesses are the labels of the top-ranked hubs, which appear in almost every label set, but a hub keeps propagating until its labels reach every vertex it dominates, and the top hubs dominate nearly the whole network, so they are among the last to finish (Figure 4). This observation motivates our *hybrid construction*: rather than waiting for the top hubs to finish, we compute their canonical labels by *PLL* before the propagation, so that the pre-insertion queries prune the redundant candidates from the first step, and a single final scan removes the little that remains. As a result, *SWPSL* produces the canonical labels up to two orders of magnitude faster than the existing methods, and among them it is the only one that indexes a web graph with 9.3 billion edges in our experiments.

Challenge 2: *It is non-trivial to maintain the labels without a dependency record while keeping them minimal.* Edge weight changes may invalidate the existing labels or make them redundant, and the weight increase is particularly difficult: the labels pruned or removed during construction do not exist in the index, but they may become necessary once the previously shorter paths get longer. *WPSL* and *BPCL* record every pruning dependency in the *PPR* [73, 75] to locate these labels, but the *PPR* can be ten times larger than the labels. *M2HL* [65] avoids the *PPR* by enumerating the distance increments one unit at a time, so its cost grows with the numeric range of the weights, and its position array also grows with the number of iterations. To maintain the labels without these heavy structures, we propose a *remove-and-recover* paradigm that records no pruning dependency at all. Instead of asking which past pruning decisions an update overturns, we ask whether each existing label is still supported: a label is kept only if one of its propagation predecessors, located by the one-byte generation step, still certifies its distance over an unchanged edge. This check may remove a few valid labels, but it is guaranteed to remove every invalid one. Then, the missing labels are recovered by a pruned propagation that skips the candidates whose vertex pairs are provably still covered by the surviving labels. Finally, since the surviving labels stay canonical, the redundancy check is restricted to the recovered labels. As a result, *SWPSL* handles 1k updates in seconds on graphs with hundreds of millions of edges, with an auxiliary structure that is only about 1/8 of the labels and 3× to 239× smaller than the *PPR*. Our contributions are summarized below:

- We propose *SWPSL*, a space-efficient 2-hop labeling for dynamic weighted small-world networks, which keeps the labels canonical and needs no costly auxiliary structure.
- We analyze the sources of the redundancy in the step-based propagation and when it can be verified, show that the removal schedules either recheck the labels repeatedly or prune late because the strongest witnesses are finalized last, and develop a

hybrid construction that precomputes the labels of the top hubs to prune the redundant candidates from the first step.
- We propose a remove-and-recover maintenance paradigm with predecessor-based invalid label detection, pruned propagation with safe pruning conditions for label recovery, and restricted redundancy removal, all with 1B per label instead of *PPR*.
- Experiments on real-world large networks validate the space saving and efficiency improvement of *SWPSL*.

The complete version [62] with proofs and additional experiments can be found in https://github.com/CSU-WRZ/SWPSL/blob/main/SWPSL_Appendix.pdf

2 PRELIMINARY

Let $G(V, E, \phi)$ be a dynamic weighted network with a vertex set V and an edge set E. Each edge $e(u, v) \in E$ has a positive weight $\phi_G(u, v)$. Let $N_G(v)$ denote the neighbors of $v \in V$, and $D(v) = |N_G(v)|$ denote its degree. Each vertex $v \in V$ is also associated with an order/rank value $r(v)$ to reflect its importance (*e.g.,* degree, closeness, betweenness centrality [14]). A path $p_G = \langle v_1, ..., v_k \rangle$ is a sequence of vertices where $(v_i, v_{i+1}) \in E$ for each $1 \leq i < k$. The length $\phi(p_G)$ of p_G is the sum of all its edge weights ($\phi(p_G) = \sum_{i=1}^{k-1} \phi_G(v_i, v_{i+1})$), and its ***hop/step*** length is the number of its edges. Let $p_{G,s,t}$ denote the set of all paths from s to t. The paths with the minimum length among $p_{G,s,t}$ are the shortest paths, denoted as $sp_{G,s,t}$, and their length is the shortest distance from s to t, denoted as $dist_G(s, t)$. We denote by $E^{\downarrow}$ and $E^{\uparrow}$ the sets of edges whose weights are decreased and increased, and by $G^{\downarrow}$ and $G^{\uparrow}$ the updated graphs after applying these two types of operations. The subscript G of the symbols is omitted when the context is clear. We assume that G is connected and undirected, and our techniques can be easily adapted to directed graphs.

Problem Statement. Given a weighted small-world network $G = (V, E, \phi)$, we aim to efficiently construct a 2-hop labeling index L and maintain L when the network changes in weight or topology.

2.1 2-Hop Labeling

2-hop labeling is the most efficient index for shortest distance queries. In general, each $v \in V$ is assigned a label set $L(v) = \{(u, dist(u, v))\}$, where the projection of $L(v)$ on the keys is called the *hub nodes* $C(v) = \{u | (u, dist(u, v)) \in L(v)\}$. We denote the distance value as $L(v)[u] = dist(u, v)$. The 2-hop labeling must satisfy the following constraint:

Constraint 1 (2-Hop Cover). For $\forall s, t \in V$, there is at least one vertex $w \in C(s) \cap C(t)$ such that $dist(s, t) = dist(s, w) + dist(w, t)$.

This constraint ensures the ***correctness***: the shortest distance between any two vertices can be correctly computed by the intersection of their label sets as $\textbf{Query}(L(s), L(t)) = dist(s, t) = \min_{x \in C(s) \cap C(t)} L(s)[x] + L(t)[x], \forall s, t \in V$, which takes $O(|L(s)| + |L(t)|)$ time when the labels are sorted by the hub order. The vertex $x \in C(s) \cap C(t)$ that minimizes the distance is the **pivot** vertex.

Constraint 2 (Canonical Labeling). In a canonical labeling L, $\forall (u, dist(u, v)) \in L(v), v \in V$, u is the vertex of the highest order among all vertices lying on any shortest path between u and v.

The *canonical* property guarantees the ***minimal*** label size, as every label entry is indispensable for the index correctness. If a label is not canonical, it is a ***redundant*** *label*, which not only increases the index size but also incurs a higher cost of construction, querying, and maintenance. The following theorem gives a sufficient condition to identify the redundant labels during construction.

Algorithm 1: WPSL Construction

```
   Input: Graph G(V, E, φ)
   Output: The 2-Hop Labeling Index L
1  Initialize L_0[v].insert(v, 0), ∀v ∈ V and h ← 1 ;
2  while L_{h-1} is not empty do
3      foreach v ∈ V in parallel do
4          foreach u ∈ N(v) do
5              foreach w ∈ C_{h-1}(u) with r(w) > r(v) do
6                  d ← φ(v, u) + L_{h-1}[u][w] ;
7                  if Q(L_{<h}[w], L_{<h}[v]) > d then
8                      L_h[v].insert(w, d);
9      h ← h + 1;
10 return L = ⋃_h L_h;
```

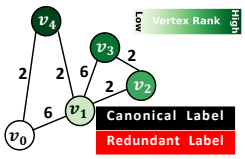
Theorem 1 (Sufficient Condition for Redundant Label Identification). A label $(u, L(v)[u]) \in L(v)$ is redundant if there exists a vertex w such that $r(w) > r(u)$, $(w, L(u)[w]) \in L(u)$, $(w, L(v)[w]) \in L(v)$, and $L(u)[w] + L(v)[w] \leq L(v)[u]$. Proof is in Appendix A.1.

2.2 Label Construction and Maintenance

Pruned Landmark Labeling (PLL) [7] follows the given vertex order r and performs a single-source shortest path (SSSP) search from each vertex $v \in V$ sequentially in this order. During the traversal, whenever the shortest distance from v to u can be correctly derived from the existing labels, the exploration to u is *pruned*. Otherwise, a new label $(v, dist(v, u))$ is added to $L(u)$. The *canonical* property is satisfied implicitly, because all the labels of the higher-order hubs have been generated before v and are used to prune $(v, dist(v, u))$ whenever a higher-order vertex lies on a shortest path between v and u. Although PLL achieves high query efficiency with the minimal index size, its construction is slow due to the strictly sequential pruned SSSP. Therefore, many efforts have been made to break the sequential constraint. **Batch Parallel Canonical Label (BPCL)** [75] runs a batch of SSSP together in parallel, but it could generate redundant labels because the labels of the current batch cannot be used for pruning within the batch, so a housekeeping step is required to remove the redundancy after each batch. **Parallel Shortest Distance Labeling (PSL)** [33, 43] further avoids SSSP entirely with parallel label propagation on unweighted graphs. It constructs the labels iteratively in a *distance-based* manner, where the labels with distance $d + 1$ are generated only after all the labels with distance d have been finalized, and being canonical is guaranteed by pruning the labels of lower orders or longer distances. Although this procedure can be extended to weighted graphs while preserving the canonical property, it suffers from low efficiency due to its reliance on fine-grained distance increments, as the number of iterations equals the largest shortest distance.

To address this limitation, **Weighted PSL (WPSL)** [68] replaces the *distance-based* propagation with a *step-based* one that iteratively generates the labels by hop length. We use L_h to denote the labels generated at step h (and $L_h[v]$ those of v), $L_{<h} = \bigcup_{i<h} L_i$ the labels generated before step h. As shown in Alg. 1, at each step h, WPSL propagates the labels generated in the previous step (L_{h-1}) to their neighbors in parallel. For each v and each neighbor $u \in N(v)$, WPSL takes every hub $w \in C_{h-1}(u)$ with a higher order than v as a candidate (line 5), and inserts a label (w, d) into $L_h[v]$ only if

(a) Example Weighted Graph



(b) PLL

	v_4-SSSP	v_3-SSSP	v_2-SSSP	v_1-SSSP	v_0-SSSP
v_0	$(v_4, 2)$				$(v_0, 0)$
v_1	$(v_4, 2)$	$(v_3, 4)$	$(v_2, 2)$	$(v_1, 0)$	
v_2	$(v_4, 4)$	$(v_3, 2)$	$(v_2, 0)$		
v_3	$(v_4, 6)$	$(v_3, 0)$			
v_4	$(v_4, 0)$				

(c) PSL/M2HL (Distance-Based)

	d = 0	d=1	d=2	d=3	d=4	d=5	d=6
v_0	$(v_0, 0)$		$(v_4, 2)$				
v_1	$(v_1, 0)$		$(v_2, 2)$ $(v_4, 2)$		$(v_3, 4)$		
v_2	$(v_2, 0)$		$(v_3, 2)$		$(v_4, 4)$		
v_3	$(v_3, 0)$						$(v_4, 6)$
v_4	$(v_4, 0)$						

(d) WPSL (Step-Based)

	0-Step	1-Step	2-Step	3-Step
v_0	$(v_0, 0)$	$(v_1, 6)$ $(v_4, 2)$	$(v_2, 8)$ $(v_3, 12)$	
v_1	$(v_1, 0)$	$(v_2, 2)$ $(v_3, 6)$ $(v_4, 2)$	$(v_3, 4)$	
v_2	$(v_2, 0)$	$(v_3, 2)$	$(v_4, 4)$	
v_3	$(v_3, 0)$		$(v_4, 8)$	$(v_4, 6)$
v_4	$(v_4, 0)$			

(e) BPCL ($\kappa = 3$)

	Batch 1 = $\{v_4, v_3, v_2\}$	Batch 2 = $\{v_1, v_0\}$
v_0	$(v_4, 2)$	$(v_0, 0)$
v_1	$(v_4, 2)$ $(v_3, 4)$ $(v_2, 2)$	$(v_1, 0)$
v_2	$(v_4, 4)$ $(v_3, 2)$ $(v_2, 0)$	
v_3	$(v_4, 6)$ $(v_3, 0)$	
v_4	$(v_4, 0)$	

(f) SWPSL Construction

	v_4-SSSP	0-Step	1-Step	2-Step
v_0	$(v_4, 2)$	$(v_0, 0)$		
v_1	$(v_4, 2)$	$(v_1, 0)$	$(v_2, 2)$ $(v_3, 6)$	$(v_3, 4)$
v_2	$(v_4, 4)$	$(v_2, 0)$	$(v_3, 2)$	
v_3	$(v_4, 6)$	$(v_3, 0)$		
v_4	$(v_4, 0)$			

Figure 2: Comparison of Different 2-Hop Labeling Constructions with Their Redundant Labels (in Red)

the new distance $d = \phi(v, u) + L_{h-1}[u][w]$ (line 6) is shorter than any distance derivable from the previous steps $L_{<h}$ (line 7). We define the label $(w, L_{h-1}[u][w]) \in L_{h-1}[u]$, where $u \in N(v)$, as the ***predecessor*** of the new label $(w, d) \in L_h[v]$, and denote it as $\mathbf{pre}[(w, d)_v] = \{(w, d - \phi(v, u))_u\}$. This predecessor relationship explicitly captures how each label in step h is derived from step $h-1$. The propagation continues until no new label is generated.

Example 1. *As shown in Fig. 2(d), the labels at 2-step are propagated from the 1-step labels.* $(v_2, 2)$, $(v_3, 6)$, $(v_4, 2)$ *in* $L_1[v_1]$ *are propagated to* v_1*'s neighbor* v_0*. Since* $Q(L_{<2}[v_0], L_{<2}[v_2]) = +\infty > \phi(v_0, v_1) + L_1[v_1][v_2] = 6 + 2 = 8$, $Q(L_{<2}[v_0], L_{<2}[v_3]) = +\infty > \phi(v_0, v_1) + L_1[v_1][v_3] = 6+6 = 12$, *two new labels* $(v_2, 8)$ *and* $(v_3, 12)$ *are inserted into* $L_2[v_0]$*. On the contrary,* $(v_4, 8)$ *is pruned since* $Q(L_{<2}[v_0], L_{<2}[v_4]) = 2$ *is smaller than* $\phi(v_0, v_1) + L_1[v_1][v_4] = 6 + 2 = 8$*. The red labels are redundant and we will analyze their sources in Section 3.1.*

Maintenance. When the edge weights change or the edges/vertices are inserted or deleted, some labels become invalid or redundant, so the index has to be maintained to preserve both the 2-hop cover constraint and the canonical property. Rebuilding the index from scratch is wasteful because each update only affects a small portion of the labels, but maintaining the labels incrementally is non-trivial: the labels pruned during construction (line 7 of Alg. 1) do not exist in L but may become necessary once the shorter paths that pruned them get longer. Therefore, *WPSL* and *BPCL* record which hub prunes each label in the *Pruning Point Record (PPR)* [68, 75], so that the pruned labels can be located when their pruning hubs are invalidated, whereas *M2HL* [65] avoids the *PPR* by deleting the invalid labels and re-propagating by distance. However, the *PPR* is several times larger than the labels, and the distance-based re-propagation suffers from the same slow convergence as *PSL* on the weighted graphs. In this work, we recover the pruned labels without either of them in Sections 4 and 5.

3 SWPSL INDEX CONSTRUCTION

In this section, we first analyze where the redundant labels of the step-based propagation come from (Section 3.1), then examine three removal schedules and show that each of them either rechecks the labels or prunes late (Section 3.2), which leads to the hybrid construction of SWPSL (Section 3.3).

3.1 Redundant Label Source

WPSL propagates the labels by hop rather than by distance, so it converges within the propagation depth H (Table 1-(e)), which is far smaller than the numeric distance range that bounds the distance-based propagation, although H is the hop length of the weighted shortest paths rather than the unweighted diameter (Appendix B.1).

However, the hop order need not follow the distance order. A label may appear before the labels that certify a shorter distance, or an equal distance through a higher-ranked hub. Consequently, a label generated in an earlier step may be pruned by a smaller-distance label constructed in a later step, leading to redundant entries in the index. Specifically, when the labels of L_{h-1} are propagated to form L_h, a higher-order hub might still be k steps away and its labels only arrive in L_{h+k}, so the condition of Theorem 1 cannot be checked at step h, and the new label $L(v)[u]$ may store neither the exact shortest distance nor the highest-order hub, *i.e.,* it is redundant with $L(v)[u] \geq dist(u, v)$. To remove them efficiently, we first classify the redundant labels into the following three types:

- **Type-1: Repetitive Hubs.** Labels share the same hub but have different distances: $(u, d_1) \in L(v)$ and $(u, d_2) \in L(v)$ but $d_1 \neq d_2$;
- **Type-2: Overestimated Distances.** A label $(u, L(v)[u])$ is *overestimated* if its recorded distance exceeds the true shortest distance, *i.e.,* $L(v)[u] > dist(u, v)$;
- **Type-3: Dominated Entries.** When $L(v)[u] = dist(u, v)$, the entry $(u, L(v)[u])$ is *dominated* if a higher-order hub x ($r(x) > r(u)$) satisfies $L(v)[x] + L(u)[x] = L(v)[u]$.

Example 2. *In Fig. 2(d), label* $(v_3, 6)$ *in* $L_1(v_1)$ *is Type-1 redundant, as a new label* $(v_3, 4)$ *with the same hub* v_3 *is inserted in the 2-step.* $(v_1, 6)$ *in* $L(v_0)$ *is Type-2 redundant, since the shorter path* $\langle v_0, v_4, v_1 \rangle$ *has distance* $4 < 6$.

Two fundamental observations govern the generation and propagation of such redundancy:

Observation 1: *Propagation from canonical labels may yield canonical or redundant labels.* Suppose $(w, d_{h-1}) \in L_{h-1}(u)$ is a canonical label, meaning that $d_{h-1} = dist(w, u)$ and w is the highest-ranked vertex on all $sp(w, u)$. By the triangle inequality, $d_h = \phi(u, v) + d_{h-1} \geq dist(w, v)$, with equality holding when $\langle w, \ldots, u, v \rangle$ forms a shortest path. **1) *Canonical Case*:** In Fig. 2(d), the label $(v_4, 4) \in L_2[v_2]$ propagated from the canonical $(v_4, 2) \in L_1[v_1]$ remains canonical. **2) *Redundant Case*:** In Fig. 2(d), $(v_4, 8) \in L_2[v_3]$ propagated from the canonical $(v_4, 2) \in L_1[v_1]$ becomes redundant.

Similarly, $(v_3, 12) \in L_2[v_0]$ is redundant, since the true shortest distance is $dist(v_0, v_3) = 8$ with the highest-ranked vertex on the path being v_4. Therefore, the redundant labels are inevitable during propagation, even when the current labels have no redundancy.

Observation 2: *Propagation from redundant labels remains redundant.* Let $(w, d_{h-1}) \in L_{h-1}(u)$ be *redundant*. We show that any propagated (w, d_h) to a neighbor v is also redundant. **1)** (w, d_{h-1}) **is Type-2:** Since (w, d_{h-1}) is overestimated ($d_{h-1} > dist_G(w, u)$), we have $d_h = \phi(u, v) + d_{h-1} > \phi(u, v) + dist_G(w, u) \geq dist_G(w, v)$, so (w, d_h) is overestimated as well and thus redundant. **2)** (w, d_{h-1}) **is Type-3:** If $dist(v, w) \neq d_h$, then the entry (w, d_h) is redundant by definition. We prove the case when $dist(v, w) = d_h$. Since (w, d_{h-1}) is dominated, there exists a $sp(w, u)$ whose highest-ranked vertex is $x \neq w$ with $r(x) > r(w)$. Then $\langle v, u, \ldots, x, \ldots, w \rangle$ is a shortest path with $r(x) > r(w) > r(v)$, so (w, d_h) is also dominated by x.

As shown in Figure 3(a), the earlier redundant labels generate more redundant labels. Therefore, the earlier the redundant labels are removed, the fewer redundant labels are generated, and the question becomes *when* a label can be verified as redundant.

3.2 Removal Schedules and Their Limitation

Type-1 repetitive hubs can be easily detected and eliminated during the construction by keeping only the label with the smaller distance for each hub. We therefore focus on Type-2 and Type-3, which cannot be decided when a label is generated because the higher-ranked hubs may not have finished propagation yet. We first establish when such a label can be verified for good, and then examine the removal schedules built on this condition.

We say that a hub w *propagates* at step h if some label $(w, \cdot)$ is generated in L_{h-1} and thus propagated to the neighbors at step h, and that w has *finished* propagation after its last propagating step. Let r_h be the highest rank among the hubs of the labels in L_h, *i.e.,* the hubs that are still active after step h, with $r_h = -\infty$ once L_h is empty.

Lemma 1 (Monotonicity of the Propagating Rank). If a hub w does not propagate at step h, it does not propagate at any step $h' > h$. Consequently, r_h is non-increasing in h. The proof is in Appendix A.3.

By Lemma 1, a hub without labels in L_h never propagates again, so all the hubs ranked above r_h have finished after step h, and the following theorem states when this suffices to verify a label.

Theorem 2 (Condition for Complete Redundancy Verification). The redundancy of the labels in $L(v)$ can be completely verified once every vertex w with $r(w) > r(v)$ has finished propagation. The proof is in Appendix A.2.

Theorem 2 has two consequences. First, after the propagation converges, one scan of the final labels identifies all the redundant labels, which is the final cleanup of SWPSL in Section 3.3. Second, during the propagation, a finished hub is a *final* witness, whose verdict by Theorem 1 never changes. The removal schedules differ in when they use the witnesses, as Figure 3 showss with the canonical labels L'_h and the redundant labels R_h generated at each step h:

1) Post-Construction Removal (PCR). A straightforward schedule [33] removes the redundancy after all the labels are constructed, so R_1, R_2, and R_3 in Figure 3(a) stay in the index until the end. Specifically, it examines each label $(u, d) \in L(v)$, and the label

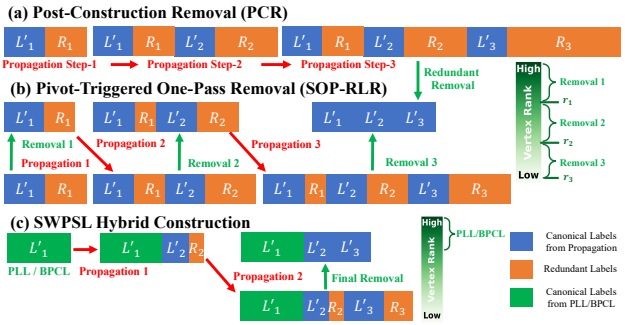


Figure 3: Removal Schedules and Hybrid Construction

is not redundant only if 1) $d = Query(L[v], L[u])$ (otherwise it is Type-2), and 2) u is the *unique* pivot hub that generates the shortest distance (otherwise it is Type-3). It checks every label once, so its complexity is $O(\sum_{v \in V} \sum_{u \in L[v]} \max(|L[v]|, |L[u]|))$, where one query takes $O(\max(|L[v]|, |L[u]|))$ time. However, the whole redundancy stays in the index throughout the construction, so the intermediate labels are several times larger than the canonical ones (Table 1-(c)), and every pruning query of every step pays for them.

2) Per-Step Removal (PSR). Alternatively, the same check can be run after every propagation step, which removes R_h as soon as its witnesses appear and keeps the intermediate index small. However, many labels are then checked repeatedly: i) the canonical labels in L'_1 are checked at every step, those in L'_2 at every step but the first, *etc*, and ii) a redundant label whose witness appears k steps later is checked k times before being removed. As each check is a distance query, PSR takes at least $\Omega(\sum_h |L_{<h}|)$ queries, even just for the canonical labels.

3) Pivot-Triggered One-Pass Removal (SOP-RLR). Theorem 2 and Lemma 1 suggest a schedule that neither waits until the end nor repeats the checks, which Figure 3(b) illustrates. The propagation partitions the ranks step by step, as the rank bar on the right shows: the hubs ranked above r_1 have finished after step 1, those in $(r_2, r_1]$ after step 2, and so on. After step h, the newly finished hubs $W_h = \{w \mid r_h < r(w) \leq r_{h-1}\}$ are used as pivots exactly once: for each $w \in W_h$ and each vertex v with $w \in C(v)$, an entry $(u, d) \in L(v)$ is removed if $r(w) > r(u)$, $w \in C(u)$, and $L(v)[w] + L(u)[w] \leq d$, which is the condition of Theorem 1 with w as the witness (Algorithm 6 in Appendix A.4). Hence, Removal 1 in Figure 3(b) deletes the labels of R_1 certified by the hubs above r_1, Removal 2 deletes R_2 with the hubs in $(r_2, r_1]$, and the labels that these pivots would certify in the later steps are never generated, because the pre-insertion queries already go through the pivots. The intervals $(r_h, r_{h-1}]$ partition the ranks, so every hub serves as a pivot exactly once and no label is checked before its witness is final, and since every redundant label meets its witness by Theorem 2, SOP-RLR also outputs the canonical labeling (Appendix A.4).

Limitation: the strongest witnesses are the last to be finalized. Figure 3(b) is the ideal case, in which a batch of hubs finishes at every step. Figure 4 shows what actually happens on BK network. Specifically, PSR shrinks the intermediate index from the first steps on, so the witnesses of most redundant labels already exist early. These witnesses are the labels of the top-ranked hubs, which lie on the shortest paths between most vertex pairs of a small-world network, so they appear in almost every label set and dominate most

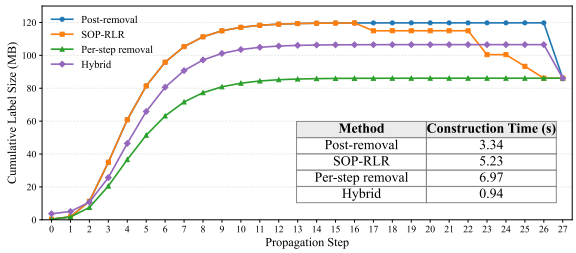


Figure 4: Intermediate Index Size and Construction Time of the Removal Schedules and Hybrid SWPSL on BK

of the redundant candidates. However, a hub keeps propagating until its labels reach every vertex it dominates, and the top hubs dominate nearly the whole network, so they are among the last to finish. On BK, the top-ranked hub propagates until step 17 of the 27 steps, so W_h is empty before that, SOP-RLR coincides with PCR through step 16, and most of its removal falls into the last four steps, which shrink the index from 119.7MB to 86.1MB. The construction times in Figure 4 expose the dilemma: PSR keeps the smallest intermediate index but is the slowest because of the rechecks, SOP-RLR checks each label once but removes little before the end, so it only adds the pivot checks to PCR, and PCR is the fastest of the three although it carries the whole redundancy, because the checks of the other two cost more than the redundancy they remove. A schedule that checks each label only once has to wait until its witness is final, and the labels of the top hubs become final last. Therefore, SWPSL makes them final before the propagation instead, which prunes the redundancy without any additional check and makes the construction many times faster than PCR, *i.e.,* WPSL (Table 1-(b)).

3.3 Hybrid SWPSL Index Construction

The top hubs are the strongest witnesses but among the last to finish, and the hybrid construction breaks this coupling by computing their labels in advance. Specifically, a pruned search of PLL from a hub w, run after the hubs ranked above w, yields exactly the canonical labels of w, which are final as soon as the search ends. Once they are present in every label set, the pre-insertion query in Algorithm 1 rejects a redundant candidate before it is generated, and by Observation 2, the redundancy that this candidate would have propagated is never generated either. Since the top hubs dominate most of the redundant candidates, a handful of searches suffices: a single PLL already removes most of the redundancy of WPSL.

Our *Hybrid SWPSL* therefore consists of three phases: PLL initialization, parallel propagation, and a final cleanup. **1) PLL initialization.** We run PLL from the top-k vertices in decreasing rank order, obtaining their canonical labels throughout the graph, as illustrated by the initialization in Figure 3(c). BPCL can parallelize this initialization when k is large. **2) Parallel propagation.** The remaining hubs then generate labels by the propagation in Algorithm 1, with the precomputed labels included in every pruning query. The top-k hubs do not propagate again, and no removal is interleaved with the propagation. **3) Final cleanup.** After the propagation converges, we scan the remaining entries once with the PCR criterion in Section 3.2: each entry (u, d) is queried against the fixed final labeling and retained only if the minimum distance equals d and u is the unique minimizing pivot, and the survivors are written to a separate output. Thus, SWPSL suppresses the redundancy during the generation and removes the residual redundancy at the end. For the maintenance in Sections 4 and 5, every label also records its *generation step* in one byte: the self label $(v, 0)$ has step 0, a propagated label in L_h has step h, *i.e.,* one more than its predecessor, and a label of a top-k hub w takes the depth of v in the pruned shortest path tree of w, which is likewise one more than the step of the label at the tree parent of v. Hence, a label of step $h \geq 1$ always has a predecessor of step $h-1$ at a neighbor, and the labels of step 1 are exactly the edge labels $(u, \phi(u, v))$.

With the v_4-SSSP in Figure 2(f), the redundant labels $(v_1, 6)$, $(v_2, 8)$, $(v_3, 12)$ of v_0 and $(v_4, 8)$ of v_3 in Figure 2(d) are never generated, as the pre-insertion queries through v_4 return smaller distances, and the final cleanup removes the survivor $(v_3, 6)$ of v_1 since $Query(L[v_1], L[v_3]) = 4 < 6$.

Figure 4 confirms this on BK: the intermediate index of the hybrid construction stays below those of PCR and SOP-RLR at every step, since the redundancy dominated by the top-k hubs is never generated, and the final cleanup removes the rest in one scan, so it builds the index in 0.94s against 3.34s (PCR), 5.23s (SOP-RLR), and 6.97s (PSR).

Theorem 3 (Correctness and Minimality of Hybrid SWPSL). The labeling output by the hybrid construction satisfies the 2-hop cover constraint and is canonical. The proof is in Appendix A.5.

Discussion. Unlike the batch-parallel PLL of BPCL and GLL [35, 75], which search from every vertex in batches, SWPSL searches only tens to hundreds of hubs and propagates the remaining labels in parallel. By Theorem 3, k affects neither the index nor the query time but only the construction time, and its selection is in Section 6.4.

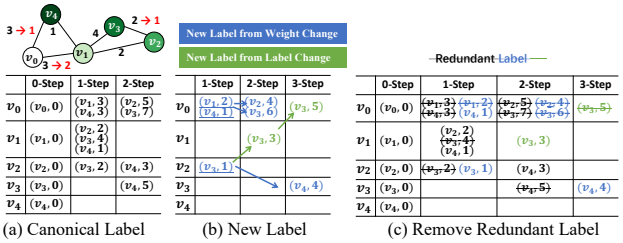
Complexity Analysis. Let $n = |V|$, $m = |E|$, p be the number of threads, H be the actual number of propagation rounds, and δ be the maximum label size of a vertex during construction, so that a query takes $O(\delta)$ time. The total construction work is $O(k(m + n \log n) + nk^2 + Hm\delta^2 + n\delta^2)$, with $O(n\delta)$ label space. The four terms account for the PLL searches, their pruning queries, the propagation, and the final cleanup, where the propagation and the cleanup are distributed across the p threads, and the PLL initialization trades its extra work for a smaller δ in the dominant term. The detailed bounds and the correctness of the parallel execution are given in Appendices B.1 and A.6.

4 SWPSL DECREASE AND INSERTION

A weight decrease or an edge insertion can only shorten the shortest paths, so the *2-hop cover* is preserved by adding new labels for the vertex pairs whose distances have decreased. In unweighted graphs, an inserted edge $e(u, v)$ sets $dist(u, v)$ to 1, and the new label $(v, 1)$ of u (assuming $r(u) < r(v)$) is propagated outward by one distance unit at a time. In weighted graphs, however, a decrease of an arbitrary amount may or may not change the distance between the endpoints, and the new labels carry arbitrary distances, so the distance-based strategies for unweighted graphs no longer apply.

4.1 New Label Generation Through Propagation

When an edge weight decreases, some vertex pairs may obtain shorter paths, which requires additional labels to preserve the 2-hop cover. To efficiently identify and propagate such updates, we design



(a) Canonical Label (b) New Label (c) Remove Redundant Label

Figure 5: Decrease Example

a two-phase propagation framework that incrementally generates the new labels without recomputing the entire index. Each label set keeps the minimum distance per hub, with absent entries valued at $+\infty$. Insertions update only strictly smaller distances. The original labeling $L = L_G$ remains read-only during propagation. As in the construction, a new label takes the generation step of its predecessor plus one, whether it is derived over a decreased edge or propagated from a new label. As shown in Algorithm 2, the process begins by detecting all the affected edges in the decreased edge set $E^{\downarrow}$. For each edge $e(a,b)$ whose new weight $\phi^{\downarrow}(a,b)$ is smaller than the previous $Q(L_G[a], L_G[b])$, a new label is inserted into $L^{\downarrow}$, and the edge is added to the active edge set $\mathcal{E}$. This forms the initial update wave, which is then expanded in two ways:

1) Weight Change Propagation (WCP): The algorithm expands the distance updates triggered by the decreased edges. For each active edge $e(a,b) \in \mathcal{E}$, WCP traverses the existing labels of both a and b to check whether a shorter distance is introduced by the new $e(a,b)$. If so, the corresponding new label is inserted into the **new label set $L^{\downarrow}$** for further propagation. Specifically, for each $h_a \in C[a]$ with $r(h_a) > r(b)$, a new candidate distance $d = \phi^{\downarrow}(a,b) + L[a][h_a]$ is computed. If $Q(L[b], L[h_a]) > d$, $e(a,b)$ offers a shorter distance to h_a, and (h_a, d) is inserted into $L^{\downarrow}[b]$. The process is symmetric for $h_b \in C[b]$.

2) Label Change Propagation (LCP): After the weight-based updates, the algorithm collects the new labels $L^{\downarrow}$ and triggers the *label-based propagation*. This phase further extends the influence of the updates by exploring whether the newly inserted labels can lead to additional shorter paths through the neighboring vertices. Similar to the index construction, the newly added labels are propagated to the neighbors round by round. For each new label $(u,d) \in L'[v]$ of the current round, LCP examines all the neighbors $v_{\text{nei}} \in N(v)$ lower than u ($r(u) > r(v_{\text{nei}})$). Accept $d' = \phi_{G^{\downarrow}}(v, v_{\text{nei}}) + d$ only if $Q(L[v_{\text{nei}}], L[u]) > d'$ and $d' < L^{\downarrow}[v_{\text{nei}}][u]$. Update the accumulated label and enqueue it in L''. Propagation stops when no label improves.

Theorem 4 (Correctness). The labeling $L \cup L^{\downarrow}$ built by Algorithm 2 satisfies the 2-hop cover constraint. The proof is in Appendix A.7.

Example 3. *As shown in Figure 5, when the weights of $e(v_0, v_1)$, $e(v_0, v_4)$ and $e(v_2, v_3)$ decrease, the reduced weights produce the new labels $(v_1, 2)$, $(v_4, 1)$ for v_0, and $(v_3, 1)$ for v_2 (blue underline). Since $(v_4, 3) \in L[v_2]$ and $\phi^{\downarrow}(v_2, v_3)$+3=4 is smaller than $Q(L[v_3], L[v_4])$=5, a new label $(v_4, 4)$ is added to $L^{\downarrow}[v_3]$ by WCP. The blue arrow points from the source (weight-changed) edge of each propagated update. The new labels collected in WCP are then propagated to the neighbors to discover additional shorter paths. For instance, $(v_3, 1) \in L^{\downarrow}[v_2]$ propagates to v_1 and yields $(v_3, 3)$ (green).*

Algorithm 2: New Label by Propagation

Input: $G(V, E, \phi)$, Labeling L_G, $E^{\downarrow}$
Output: L, $L^{\downarrow}$

Initialize $L \leftarrow L_G$, $L^{\downarrow} \leftarrow \emptyset, \mathcal{E} \leftarrow \emptyset$;
foreach $\phi^{\downarrow}(a,b) \in E^{\downarrow}$ **do**
  $d_{ab} \leftarrow Q(L_G[a], L_G[b])$;
  if $d_{ab} > \phi^{\downarrow}(a,b)$ **then**
    $L^{\downarrow}[a].insert(b, \phi^{\downarrow}(a,b)), \mathcal{E}.insert(\{a,b\})$; ▷ $r(b) > r(a)$
foreach $(a,b) \in \mathcal{E}$ **do**
  WCP$(a, b, L_G, L^{\downarrow})$; ▷ Weight Change Propagation
$L' \leftarrow L^{\downarrow}$
while L' *is not empty* **do**
  $L'' \leftarrow \emptyset$
  foreach $v \in V$ **do**
    foreach $(u,d) \in L'[v]$ **do**
      LCP$(v, u, d, L_G, L^{\downarrow}, L'')$ ▷ Label Change Propagation
  $L' \leftarrow L''$; ▷ New labels generated in the current round
return L, $L^{\downarrow}$;

Function WCP$(a, b, L, L^{\downarrow})$:
  foreach $h_a \in C[a]$ *with* $r(h_a) > r(b)$ **do**
    $d \leftarrow \phi^{\downarrow}(a,b) + L[a][h_a]$;
    if $Q(L[b], L[h_a]) > d$ **then**
      $L^{\downarrow}[b].insert(h_a, d)$;
  foreach $h_b \in C[b]$ *with* $r(h_b) > r(a)$ **do**
    $d \leftarrow \phi^{\downarrow}(a,b) + L[b][h_b]$;
    if $Q(L[a], L[h_b]) > d$ **then**
      $L^{\downarrow}[a].insert(h_b, d)$;

Function LCP$(v, u, d, L, L^{\downarrow}, L'')$:
  foreach $v_{nei} \in N(v)$ *with* $r(u) > r(v_{nei})$ **do**
    $d' \leftarrow \phi_{G^{\downarrow}}(v, v_{nei}) + d$;
    if $Q(L[v_{nei}], L[u]) > d'$ *and* $d' < L^{\downarrow}[v_{nei}][u]$ **then**
      $L^{\downarrow}[v_{nei}].insert(u, d')$;
      $L''[v_{nei}].insert(u, d')$;

4.2 Redundant Label Removal

After the new labels are generated, both the new label set $L^{\downarrow}$ and the old label set L may have redundancy. Although these redundant labels can answer the queries correctly, their continuous accumulation during the updates leads to larger index sizes, which degrade the query and update performance. Therefore, this section introduces our redundancy elimination method to maintain a canonical and compact label.

For an entry (u,d) at v, the query on $L \cup L^{\downarrow}$ splits into four combinations: Q_1 uses $(L[u], L[v])$, Q_2 uses $(L^{\downarrow}[u], L[v])$, Q_3 uses $(L[u], L^{\downarrow}[v])$, and Q_4 uses $(L^{\downarrow}[u], L^{\downarrow}[v])$. Each $Q_i(u,v) = (q_i, P_i)$ returns the minimum distance and its pivots. Define $R_i(u,v,d) \equiv (q_i < d) \vee (q_i = d \wedge P_i \setminus \{u\} \neq \emptyset)$.

This preserves equality witnessed only by u but allows a shorter entry with the same hub to replace an old one. Empty intersections return $(+\infty, \emptyset)$. The four combinations cover the merged query, whose redundancy test is $\bigvee_i R_i$. Throughout filtering, L and $L^{\downarrow}$ are read-only and $A^{\downarrow} = \{v \mid L^{\downarrow}[v] \neq \emptyset\}$ is fixed. Survivors are written to a separate output L'.

(1) New labels $L^{\downarrow}$: New entries satisfy $q_1 > d$ by pre-insertion checks. Test R_4, R_2, R_3 and copy an entry to L' only if all are false.

(2) Old labels L: Old canonical entries have $q_1 = d$ and $P_1 = \{u\}$, so neither phase needs Q_1. The fixed affected set determines the remaining tests.

If neither endpoint is affected (Case 1), copy the entry to L'. If only v (Case 2) or only u (Case 3) is affected, test R_3 or R_2. If both are affected (Case 4), test R_4, R_2, R_3. Copy the entry only if all applicable tests are false.

Theorem 5 (Correctness and Minimality). The labeling index L' generated by Algorithm 3 satisfies both the 2-hop cover constraint and the canonical property. The proof can be found in Appendix A.8.

Example 4. *As shown in Figure 5(c), $(v_3, 6) \in L^{\downarrow}[v_0]$ is removed as $q_4(v_3, v_0) = 5 < 6$, while $(v_4, 1)$ is retained. The old labels are then checked against the fixed affected set. For instance, $(v_1, 3) \in L[v_0]$*

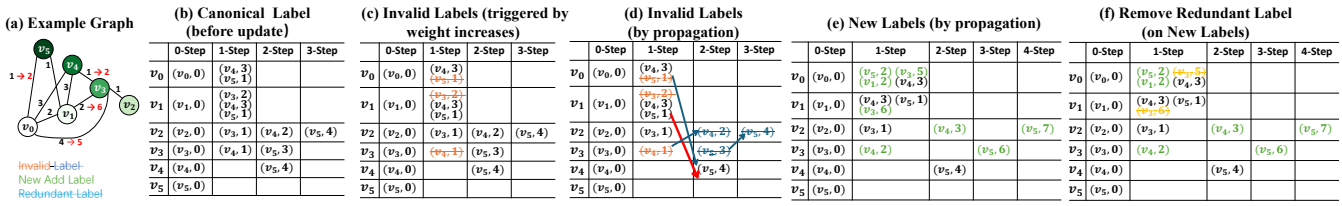


Figure 6: SWPSL Increase Maintenance Example

Algorithm 3: Cross-Validation Redundancy Removal

Input: Graph $G(V, E)$, Old Labeling L, New Labeling $L^{\downarrow}$
Output: Canonical labeling L'

$L' \leftarrow \emptyset, A^{\downarrow} \leftarrow \{v \mid L^{\downarrow}[v] \neq \emptyset\}$;
foreach $v \in V$ ***in parallel*** **do**
  foreach $(u, d) \in L^{\downarrow}[v]$ **do**
    if $R_4(u, v, d)$ ***or*** $R_2(u, v, d)$ ***or*** $R_3(u, v, d)$ **then** continue ;
    $L'[v].insert(u, d)$;
foreach $v \in V$ ***in parallel*** **do**
  foreach $(u, d) \in L[v]$ **do**
    if $u \notin A^{\downarrow}$ ***and*** $v \notin A^{\downarrow}$ **then**
      $L'[v].insert(u, d)$;
      continue;
    else if $u \notin A^{\downarrow}, v \in A^{\downarrow}$ **then**
      if $R_3(u, v, d)$ **then** continue ;
    else if $u \in A^{\downarrow}, v \notin A^{\downarrow}$ **then**
      if $R_2(u, v, d)$ **then** continue ;
    else if $u \in A^{\downarrow}, v \in A^{\downarrow}$ **then**
      if $R_4(u, v, d)$ ***or*** $R_2(u, v, d)$ ***or*** $R_3(u, v, d)$ **then** continue ;
    $L'[v].insert(u, d)$;
return L';
Function $R_i(u, v, d)$:
  $(q_i, P_i) \leftarrow Q_i(u, v)$;
  return $q_i < d$ ***or*** $(q_i = d$ ***and*** $P_i \setminus \{u\} \neq \emptyset)$;
// Queries below return the distance and all minimizing pivots.
Function $Q_2(u, v)$: **Return** $Query(L^{\downarrow}[u], L[v])$;
Function $Q_3(u, v)$: **Return** $Query(L[u], L^{\downarrow}[v])$;
Function $Q_4(u, v)$: **Return** $Query(L^{\downarrow}[u], L^{\downarrow}[v])$;

is removed as $q_3(v_1, v_0) = 2 < 3$*, while the canonical* $(v_2, 2) \in L[v_1]$ *remains.*

Complexity Analysis. Let H_d be the number of LCP rounds and m_d be the maximum sum of degrees of the vertices that receive new labels in one round. Algorithm 2 takes $O(|E^{\downarrow}|\delta^2 + H_d m_d \delta^2)$ time, and Algorithm 3 takes $O(|L^{\downarrow}|\delta)$ time for the new labels and $O(n\delta^2)$ time for the old labels in the worst case, where the pairs without any affected endpoint (Case 1) are skipped without any query. Both algorithms are parallelized over vertices (Appendix A.6), the extra space is the temporary $L^{\downarrow}$ that is released after the update, and the detailed analysis can be found in Appendix B.2.

Edge and Vertex Insertion. Both are reduced to the weight decrease and handled by Algorithms 2 and 3 without modification (Appendix C.1): an inserted edge $e(a, b)$ is treated as an existing edge whose weight decreases from infinity to its new weight, and an inserted vertex v is initialized with $L(v) = \{(v, 0)\}$ and the lowest rank before its incident edges are inserted.

5 SWPSL INCREASE AND DELETION

As analyzed in [68], the weight increase is the biggest obstacle to efficient maintenance: some canonical labels no longer reflect the true shortest distances, while the labels pruned or removed as redundant during construction are absent from the index but may now become canonical. The state-of-the-art *Pruning Point Record (PPR)* [68, 75] locates the latter by recording every pruning dependency during construction, which costs substantial memory and complex traversals over the dependency structure. In this section, we propose a *Remove-and-Recover* mechanism that handles the weight increases without PPR.

Generation Step. The only information the mechanism relies on is the generation step of each label, which is assigned uniformly in the construction (Section 3.3) and the maintenance: the self labels have step 0, and every other label takes the step of the predecessor it is derived from plus one, whether it is generated by the propagation, by the PLL searches, by the decrease (Section 4), or by the recovery below, whose round counter t is never stored. Consequently, the labels of step 1 are exactly the edge labels, on which the completeness of the removal below relies (Theorem 6), and a label of step $h > 1$ keeps a predecessor of step $h - 1$ at a neighbor unless an update has changed its path. The step is never recomputed afterwards: a label whose recorded predecessors have disappeared, including an edge label whose edge has increased, is treated as unsupported and removed conservatively, and the recovery regenerates it with its new step if it is still needed, so the invariant holds after every batch of updates. Since the steps follow the hop lengths of the shortest paths, one byte suffices on all our datasets (maximum step 142, Table 1-(e)) and two bytes otherwise.

5.1 Remove Invalid Labels

When an edge weight increases, some previously canonical labels $(u, L(v)[u])$ may *underestimate* the updated shortest distance, *i.e.*, $L(v)[u] < dist_{G^{\uparrow}}(v, u)$, thereby violating the 2-hop cover constraint. We call any label that underestimates its updated shortest distance an **invalid label**; otherwise, it is a **valid label**. To preserve the correctness with a low maintenance cost, Algorithm 4 identifies and removes all the invalid labels, starting from the edge labels of the increased edges. In unweighted graphs, deleting $e(a, b)$ always invalidates the edge label $(b, 1) \in L(a)$ (assuming $r(a) < r(b)$), whereas an increase of $e(a, b)$ in a weighted graph does not necessarily alter any shortest path, so an edge label may stay valid although its edge has increased. Nevertheless, the following lemma shows that these edge labels are the only labels of step 1 that can become invalid.

Lemma 2 (Increase Trigger Condition). Given G, its canonical labeling L, and the increased edges $E^{\uparrow}$, a label of step 1 becomes invalid only if it is the edge label $(b, \phi_G(a, b)) \in L_1[a]$ (assuming $r(a) < r(b)$) of an increased edge $e(a, b) \in E^{\uparrow}$. The proof is in Appendix A.9.

Therefore, for every increased edge $e(a, b)$, Algorithm 4 first marks its edge label $(b, \phi_G(a, b))$ for deletion if it exists, without checking whether the distance between a and b actually changes. This conservative removal costs one lookup per edge and keeps the

Algorithm 4: Remove Invalid Labels

Input: $G(V, E, \phi)$, Labeling L_G with generation steps, $E^{\uparrow}$
Output: Pruned labeling L, affected vertices $\mathcal{A}^{\uparrow}$, affected max-rank Max_r

1 **Initialize** $\mathcal{A}^{\uparrow} \leftarrow \emptyset$ and the deleted hub set $D[v] \leftarrow \emptyset, \forall v \in V$;
2 $maxStep \leftarrow$ maximum recorded generation step in L_G;
3 // Collect requests in D and publish at synchronization.
4 **foreach** $e(a, b) \in E^{\uparrow}$ *with* $r(a) < r(b)$ ***in parallel*** **do**
5 **if** $(b, \phi_G(a, b)) \in L_{G,1}[a]$ **then**
6 mark $(b, \phi_G(a, b))$ in $L_G[a]$ as deleted; ▷ Edge Label Removal (Lemma 2)
7 $D[a].insert(b)$;
8 Synchronize deletion marks;
9 **for** $h \leftarrow 2$ **to** $maxStep$ **do**
10 **foreach** $v \in V$ ***in parallel*** **do**
11 **foreach** $(u, d) \in L_{G,h}[v]$ *with* $u \notin D[v]$ **do**
12 **if** CheckPredecessor$(u, d, v, h) = False$ **then**
13 mark (u, d) in $L_G[v]$ as deleted; ▷ Predecessor Validation
14 $D[v].insert(u)$;
15 Synchronize deletion marks;
16 $L[v] \leftarrow \{(u, d) \in L_G[v] \mid u \notin D[v]\}, \forall v \in V$;
17 **foreach** $v \in V$ *with* $D[v] \neq \emptyset$ **do**
18 $\mathcal{A}^{\uparrow} \leftarrow \mathcal{A}^{\uparrow} \cup \{v\}, Max_r[v] \leftarrow \max_{w \in D[v]} r(w)$;
19 **return** $L, \mathcal{A}^{\uparrow}, Max_r$;
20 **Function** CheckPredecessor(u, d, v, h):
21 **foreach** $x \in N(v)$ ***with*** $e(v, x) \notin E^{\uparrow}$ **do**
22 **if** $(u, d - \phi(v, x)) \in L_{G,h-1}[x]$ ***and*** $u \notin D[x]$ **then return** $True$;
23 **return** $False$;

step invariant of Section 3.3: every surviving label of step 1 is the edge label of an unchanged edge, so it is still an edge label after the update. Self-labels $(v, 0)$ remain valid and are retained as recovery seeds. Conservatively removed valid entries are restored if needed.

Detection reads a fixed snapshot L_G. The set $D[v]$ records hubs marked for deletion, which *CheckPredecessor* treats as absent. Concurrent marks are merged by union after the edge phase and after each generation step, before the next phase reads them. Physical deletion follows detection.

Example 5. *In Figure 6(a), the weights of the edges $e(v_0, v_3)$, $e(v_0, v_5)$, $e(v_1, v_3)$, and $e(v_3, v_4)$ increase. Three of them have edge labels in Figure 6(b), namely $(v_5, 1) \in L[v_0]$, $(v_3, 2) \in L[v_1]$, and $(v_4, 1) \in L[v_3]$, which are marked for deletion in Figure 6(c), whereas $e(v_0, v_3)$ has no edge label because v_4 and v_5 lie on other shortest paths between v_0 and v_3, and $(v_4, 3) \in L[v_0]$ is kept as its edge $e(v_0, v_4)$ is unchanged.*

After the initial marking phase, we verify each remaining label step by step through all the label generation steps from $h = 2$ up to $maxStep$, where $maxStep$ is the largest generation step recorded in the current input index L_G, so every label is verified at its own step and no label is missed. For each $(u, d) \in L_{G,h}[v]$ generated at step h, the procedure *CheckPredecessor* examines whether this label can still be supported by an unmarked predecessor in $L_{G,h-1}$ according to the following lemma.

Lemma 3 (Supportive Predecessor Validation). A label $(w, d) \in L_{G,h}[v]$ remains valid in $G^{\uparrow}$ if there exists at least one valid predecessor label $(w, d') \in L_{G,h-1}[u]$ such that $d = \phi(v, u) + d'$ and $e(v, u) \notin E^{\uparrow}$. The proof is in Appendix A.10.

Specifically, for each neighbor $x \in N(v)$ such that $e(v, x) \notin E^{\uparrow}$, a predecessor supports (u, d) only if $(u, d - \phi(v, x)) \in L_{G,h-1}[x]$ and $u \notin D[x]$. If no such predecessor exists, mark (u, d) for deletion.

Example 6. *In Figure 6(d), $(v_5, 3) \in L_G[v_3]$ is invalid since its only predecessor $(v_5, 1)_{v_1}$ lies behind the increased edge $e(v_1, v_3)$, and $(v_4, 2) \in L_G[v_2]$ is invalid since its only predecessor $pre[(v_4, 2)_{v_2}] = \{(v_4, 1)_{v_3}\}$ is deleted due to the increase of $e(v_3, v_4)$ (blue). Although $(v_5, 1) \in L_G[v_0]$, one predecessor of $(v_5, 4) \in L_G[v_4]$, becomes invalid, $(v_5, 4) \in L_G[v_4]$ remains valid as it is supported by another valid predecessor $(v_5, 1) \in L_G[v_1]$ (red). At step 3, $(v_5, 4) \in L_G[v_2]$ is removed in turn, as its only predecessor $(v_5, 3)_{v_3}$ has been deleted.*

Theorem 6 (Invalid Label Removal Completeness). Algorithm 4 removes all the invalid labels in L. The proof is in Appendix A.11.

Lemma 3 is sufficient but not necessary, so Algorithm 4 may also remove a few valid labels, which is safe because no invalid label survives (Theorem 6) and the recovery in Section 5.2 regenerates the missing ones (Appendix A.11).

5.2 Recover Pruned and Removed Labels

After removing the invalid labels, the remaining L may violate the 2-hop cover, as some of the labels pruned or removed during construction have to be recovered. Continuing the propagation from all the remaining labels would recover them, but most labels have already propagated during construction and nothing has changed around them. Therefore, we confine the propagation to the following *affected labels*:

Definition 1 (Affected Vertices and Max-Rank). Let L be the label after deleting all the invalid labels (Algorithm 4). A vertex v is *affected*, denoted as $v \in \mathcal{A}^{\uparrow}$, iff $L(v)$ removes at least one label during the deletion. For $v \in \mathcal{A}^{\uparrow}$, its *Affected Max-Rank* $Max_r[v]$ is the highest-ranked hub in the deleted label set of $L[v]$.

Then we propose to recover these labels by the pruned propagation on the affected labels, as detailed in Algorithm 5. It proceeds in rounds, where $L_{(t)}$ denotes the labels generated in round t, and the whole pruned labeling serves as the initial frontier $L_{(0)}$. In each round t, every vertex v collects the candidate hubs from the frontier labels $L_{(t-1)}[u]$ of its neighbors (line 6) from two complementary sources:

(1) Weight-Update Propagation: For each edge $e(v, u) \in E^{\uparrow}$ whose weight has increased, the candidate hubs from u are checked with the increased weight, as their distances through $e(v, u)$ may have changed (line 9).

(2) Label-Triggered Propagation: For the unaffected edges $e(v, u) \notin E^{\uparrow}$, if either v or the candidate hub w has been marked as affected, we need to check whether the labels of the neighbors can generate new labels for v (lines 7-10), while the following two pruning conditions allow the propagation to skip the candidates that are already covered by the remaining labels (line 7).

Pruning Condition 1: *If $v \notin \mathcal{A}^{\uparrow}$ and $w \notin \mathcal{A}^{\uparrow}$ with $r(w) > r(v)$, then the candidate (w, d) can be safely pruned.*

Pruning Condition 2: *If $v \in \mathcal{A}^{\uparrow}$ and $w \notin \mathcal{A}^{\uparrow}$ with $r(w) > Max_r[v]$, then the candidate label (w, d) can be safely pruned.*

Both conditions only skip the pairs that are still covered by the surviving labels: neither v nor w loses any label in Pruning Condition 1, and every hub of v ranked above $Max_r[v]$, including the pivot between v and w, survives the deletion in Pruning Condition 2. The proofs can be found in Appendices A.12 and A.13.

For the candidates that survive the pruning, the new distance $d = \phi^{\uparrow}(v, u) + L[u][w]$ is checked against the current labeling (line 10), and the new label is inserted into $L_{(t)}[v]$ with the step of its predecessor $(w, L[u][w])$ plus one, where the insertion keeps the smaller distance when the same hub arrives from several neighbors. After each round, $L_{(t)}$ is merged into L for the queries of the next

Algorithm 5: Label Recovery by Pruned Propagation

Input: Graph $G(V, E, \phi^{\uparrow})$, pruned labeling L, affected vertices $\mathcal{A}^{\uparrow}$, affected max-rank Max_r
Output: The remaining labels L_{old} and the recovered labels L_{new}

1 $L_{old} \leftarrow L, L_{new} \leftarrow \emptyset, L_{(0)} \leftarrow L, t \leftarrow 1$;
2 **while** $L_{(t-1)}$ *is not empty* **do**
3 $L_{(t)} \leftarrow \emptyset$;
4 **foreach** $v \in V$ ***in parallel*** **do**
5 **foreach** $u \in N(v)$ **do**
6 **foreach** $w \in C_{(t-1)}(u)$ ***with*** $r(w) > r(v)$ **do**
7 **if** $e(v,u) \notin E^{\uparrow}$ ***and*** $w \notin \mathcal{A}^{\uparrow}$ ***and*** $(v \notin \mathcal{A}^{\uparrow}$ ***or*** $r(w) > Max_r[v])$ **then**
8 **continue** ▷ Pruning Conditions 1 and 2
9 $d \leftarrow \phi^{\uparrow}(v,u) + L[u][w]$;
10 **if** $Q(L[v], L[w]) > d$ **then** $L_{(t)}[v]$.insert(w, d) ;
11 $L \leftarrow L \cup L_{(t)}, L_{new} \leftarrow L_{new} \cup L_{(t)}, t \leftarrow t+1$;
12 **return** L_{old}, L_{new};

round and accumulated into L_{new} (line 11). Moreover, since the candidates are taken only from the frontier $L_{(t-1)}$ rather than from the whole labeling, a label is propagated exactly once, in the round after it is generated. The propagation terminates when no new label is generated. Here L_{old} denotes the labels that survive Algorithm 4 (*i.e.,* the input of this algorithm) and L_{new} the labels generated during the recovery, which are kept separate for the redundancy removal in Section 5.3.

Example 7. *Figure 6(e) shows the label recovery. In round 1, the vertices acquire labels from their neighbors:* v_1 *inserts* $(v_3, 6)$ *and* v_3 *inserts* $(v_4, 2)$. *In round 2, these labels propagate further:* v_2 *obtains* $(v_4, 3)$ *via* $(v_4, 2) \in L[v_3]$. *According to the original WPSL propagation rule,* $(v_5, 1) \in L[v_1]$ *would also be propagated to* $L[v_4]$. *However, with our Pruning Condition 1, neither* v_4 *nor* v_5 *belongs to* $\mathcal{A}^{\uparrow}$, *so this propagation can be safely pruned.*

Theorem 7 (Correctness). The labeling $L_{old} \cup L_{new}$ output by Algorithm 5 satisfies the 2-hop cover constraint. The proof is in Appendix A.14.

5.3 Redundant Label Removal

It should be noted that the above pruning conditions in the recovery phase avoid some redundancy, but still cannot avoid all. Although we can run the redundancy removal as in construction, it contains many unnecessary checks. In fact, we can further improve it by only testing the recovered labels L_{new}, since the remaining labels L_{old} after the deletion are already non-redundant, as proved by the following theorem.

Theorem 8. The labels L_{old} remaining after Algorithm 4 contain no redundant label. The proof is in Appendix A.15.

With fixed inputs L_{old} and L_{new}, apply the new-label phase of Algorithm 3, testing R_4, R_2, R_3. Pre-insertion checks already exclude candidates covered by L_{old}, so Q_1 is unnecessary. The final index is $L_{old} \cup \widehat{L}_{new}$, where $\widehat{L}_{new}$ contains the surviving recovered entries. All old entries are retained.

Example 8. *In Figure 6(f),* $(v_3, 5) \in L[v_0]$ *is dominated by* $(v_4, 3) \in L[v_0]$ *and* $(v_4, 2) \in L[v_3]$, *while* $(v_3, 6) \in L[v_1]$ *is Type-2 redundant as* $dist_{G^{\uparrow}}(v_1, v_3) = 5 \neq 6$.

Complexity Analysis. Let H_i be the number of recovery rounds and m_i be the maximum sum of degrees of the vertices that receive new labels in one round. The edge label removal, the predecessor validation, the pruned propagation, and the redundancy removal take $O(|E^{\uparrow}|)$, $O(Hm\delta)$, $O(H_i m_i \delta^2)$, and $O(|L_{new}|\delta)$ time, where $H = maxStep$ is the maximum recorded generation step in the input index and $|L_{new}| = O(H_i m_i \delta)$, and the edge phase is parallelized over the updated edges, while the other phases are parallelized over the vertices with p threads (Appendix A.6). The extra space is the one-byte generation step of each label together with the temporary L_{new}, which is released after the update, so SWPSL requires no persistent structure like the PPR. The detailed analysis can be found in Appendix B.3.

Edge and Vertex Deletion. Both are reduced to the weight increase and handled by Alg.4 and 5 without modification (Appendix C.2): a deleted edge is treated as an edge whose weight increases to ∞, and a deleted vertex as the deletion of all its incident edges.

6 EXPERIMENT

6.1 Experiment Setup

Dataset. We use 13 real-world graphs (Table 1-(a)), each converted into an undirected graph with its largest connected component. The first 7 graphs have no native weights, so we follow [68–70, 75] to assign each edge a random weight inversely proportional to the larger degree of its endpoints and use them to evaluate the scalability. Since this weighting concentrates the shortest paths on the high-degree vertices and makes the redundancy of WPSL heavier. The last 6 graphs have native weights.

Baselines. We compare with **1) *WPSL*** [68], the step-based propagation with redundant labels and PPR; **2) *BPCL*** [75], the batch PLL without redundant labels and with PPR; and **3) *M2HL*** [65, 66], the distance-based propagation without redundant labels that updates with a distance position array. The earlier maintenance methods for the unweighted PLL [8, 20] are outperformed by M2HL by 10000× [65], the static GLL and PLaNT [35] are outperformed by BPCL [75], the tree decomposition-based methods [23, 67, 76] cannot construct on small-world networks, so we do not compare them. We only report BatchHL's [24] query time.

Update Set. We randomly select 1k edges, whose weights are halved to decrease and doubled to increase. For the topology updates, we randomly delete and insert 1k edges or vertices.

Environment. All the algorithms are implemented in `C++` with `-O3` optimization and `OpenMP`, share the same vertex ordering, and run on an Ubuntu 20.04 server with two 32-core Intel Xeon Platinum 8375C 2.9GHz CPUs and 2TB of memory with 64 threads by default. The vertex identifiers and distances are 4-byte `int` and the step numbers are 1 byte. A result is reported as "INF" or "OOM" if an algorithm fails to finish within 12 hours or exceeds the memory.

6.2 Indexing and Query Performance

Memory Consumption. Table 1-(c) reports the label sizes together with the auxiliary maintenance structures. BPCL, M2HL, and SWPSL generate the same canonical labels, whereas 76.0% to 94.9% of the labels of WPSL are redundant on the synthetic-weight graphs (up to 20× the canonical labels) and 21.6% to 54.8% on the native-weight graphs. For the auxiliary structures, the PPR of BPCL (the minimal one, so that of WPSL is not reported) is still up to 10× larger than the labels, and the position array of M2HL exceeds 100× the label size when the maximum distance is large, whereas SWPSL only stores one byte per label for the generation step (all below 200), which is about 1/8 of the canonical labels and 3× to 239× smaller than the PPR.

Table 1: Dataset and Index Performance

Name	Dataset	(a) Statistics			(b) Construction Time (s)				(c) Index Size (MB)						(d) Query Time (μs)			(e) Maximum Propagation Length	
									Canonical Label	Redundant Label		Auxiliary Structure							
		$\|V\|$	$\|E\|$	D_{max}	WPSL	BPCL	M2HL	SWPSL	BPCL/M2HL/SWPSL	WPSL	Re Ratio	WPSL/BPCL	M2HL	SWPSL	WPSL	BPCL/M2HL/SWPSL	BatchHL	WPSL/SWPSL (steps)	M2HL (distance)
WT	Wiki-Talk[36]	2,388,953	9,313,364	100,029	522.24	2062.94	OOM	**11.65**$_{k=50}$	227.60	1813.91	87.5%	82.31	OOM	**28.45**	3.46	0.29	18,310	8	OOM
SK	Skitter[36]	1,694,616	22,188,418	35,455	308.91	1037.95	OOM	**11.67**$_{k=50}$	230.00	2377.07	90.0%	525.42	OOM	**28.75**	2.36	0.39	12,470	23	OOM
PO	Pokec[36]	1,632,803	44,603,928	14,854	815.75	981.52	5547.72	**21.81**$_{k=50}$	636.28	11727.78	94.6%	7380.54	263465.63	**79.53**	9.03	0.83	12,622	11	42,298
LJ	LiveJournal1[36]	4,843,953	85,691,368	20,333	3022.69	8046.01	OOM	**70.71**$_{k=10}$	1178.13	22987.37	94.9%	5170.66	OOM	**147.27**	5.43	0.63	59,949	13	OOM
WI	Wikipedia_fr[34]	3,333,272	200,923,676	399,358	1569.63	4011.39	18724.57	**73.37**$_{k=10}$	2923.29	12201.56	76.0%	31582.80	474374.31	**365.41**	4.14	1.34	37,673	6	37,306
HO	Hollywood[1]	1,917,070	228,562,202	13,107	2047.49	2486.74	20874.73	**102.60**$_{k=100}$	4218.43	34255.04	87.7%	35597.90	333555.14	**527.30**	15.89	1.98	21,689	9	45,610
UK	uk-union[1]	130,831,972	9,306,348,820	6,366,526	OOM	INF	OOM	**4945.31**$_{k=10}$	92670.60	-	-	-	OOM	**11583.82**	-	2.86	-	142	OOM
BK	Brightkite[36]	48,129	377,932	1,098	5.88	10.22	107.02	**0.94**$_{k=10}$	86.10	119.76	28.1%	75.89	12626.40	**10.76**	3.56	2.22	3,199	27	68,771
YF	Yelp_FGRec[4]	22,651	520,790	2,125	1.10	1.98	14.81	**0.17**$_{k=10}$	13.55	29.94	54.8%	70.37	2087.07	**1.69**	1.49	0.75	2,014	19	24,153
GO	Gowalla[36]	95,848	885,692	9,962	12.91	21.57	324.58	**2.13**$_{k=50}$	179.76	253.47	29.1%	210.88	24350.30	**22.47**	3.16	2.39	8,761	27	66,597
IC	Interactiv_SF10[22]	60,928	3,125,244	1,514	100.31	212.45	472.26	**10.38**$_{k=200}$	401.60	659.70	39.1%	1689.47	13738.00	**50.20**	12.38	7.96	11,527	19	59,107
WIT	Wikipedia_Talk[3]	1,091,742	5,573,526	141,951	160.97	528.02	81.58	**16.28**$_{k=100}$	941.68	1246.17	24.4%	3360.27	6513.54	**117.71**	4.70	1.30	7,144	7	1,563
TSO	tags-stack-overflow[2]	49,931	8,294,590	23,395	16.50	40.15	2376.82	**1.26**$_{k=10}$	141.77	180.90	21.6%	4228.77	89380.90	**17.72**	4.46	3.54	1,008	4	469,260

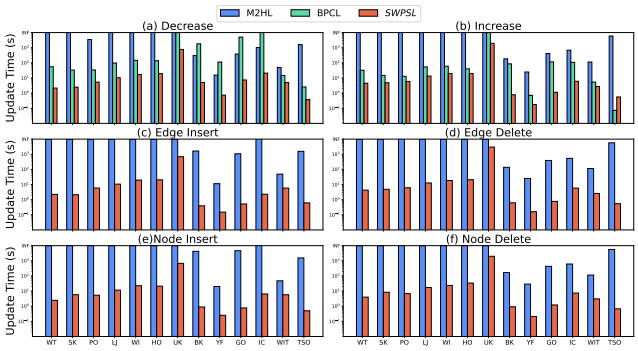


Figure 7: Index Maintenance Time (s), 1k Updates

Maximum Propagation Length. Table 1(e) reports the maximum propagation extent, measured in steps for WPSL/SWPSL and in distance units for M2HL. WPSL/SWPSL require 4–27 steps on all datasets except UK, which requires 142. In contrast, M2HL advances the propagation distance by one unit per iteration, reaching 1,563–469,260 distance units on datasets where it completes. This fundamental gap in convergence speed is the primary driver of the construction time differences presented next.

Index Construction Time. As shown in Table 1-(b), SWPSL is 20× to 45× faster than WPSL on the synthetic-weight graphs and 6× to 13× faster on the native-weight graphs, where the smaller gap is consistent with the lower redundancy of the native weights, and it is also much faster than BPCL and M2HL, which are bounded by the pruned searches and the distance-based iterations, respectively. On UK with 9.3 billion edges, WPSL and M2HL run out of memory and BPCL fails to finish within the time limit, whereas SWPSL constructs the canonical index in 4,945s with 11.6GB of auxiliary data for a 92.7GB index. Therefore, among the compared methods, SWPSL is the only one that builds the canonical labels on a graph with billions of edges.

Query Efficiency. We randomly sample 100k OD pairs and report the average query time. As shown in Table 1-(d), the canonical indexes are 10× faster than the redundant ones.

6.3 Update Efficiency

Weight Changes. Figure 7(a)-(b) report the total maintenance time of 1k weight updates. M2HL takes orders of magnitude longer for the same reason as its long construction time, and it fails on the large graphs where its index cannot be built, while SWPSL is faster than BPCL for the decreases and comparable for the increases. Figure 8 further varies the update number from 1 to 100k, with the construction time of SWPSL as the horizontal reference line, where M2HL is omitted since even a single decrease on PO takes 1,357s. As

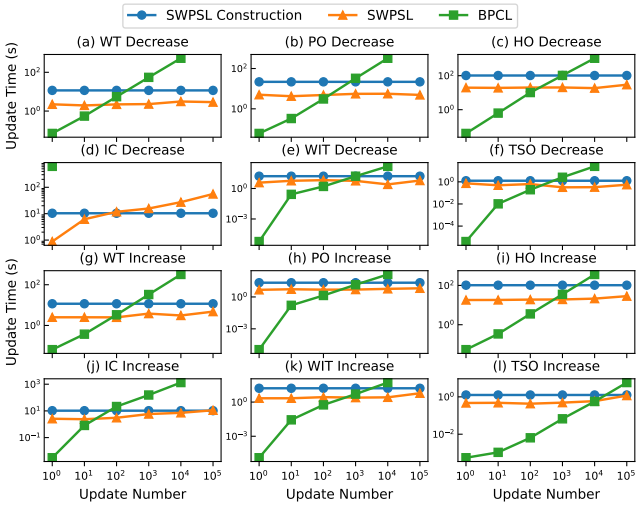


Figure 8: Maintenance of Different Update Numbers

BPCL maintains the labels one by one, it is very fast for a handful of updates, but it becomes slower than SWPSL once the updates exceed a few hundred (decrease) or a few thousand (increase), and it eventually takes longer than the construction of SWPSL. On the contrary, the parallel propagation keeps the maintenance time of SWPSL nearly stable under 10k updates and consistently below the reconstruction, and SWPSL is still several times faster where the log-scale curves look close. Therefore, the trade-off between BPCL and SWPSL is a memory-latency one: BPCL answers a handful of updates faster, but it has to keep a PPR that is 3× to 239× larger than the auxiliary structure of SWPSL (up to 35.6GB on HO) permanently in memory, whereas SWPSL keeps only one byte per label and wins once the updates arrive in batches of a few hundred, which is the common case when the updates are applied periodically.

Topology Changes. Figure 7(c)-(f) report the edge and vertex insertions and deletions, which are reduced to the weight changes (Appendix C). SWPSL outperforms M2HL on all the datasets for all the four operations, often by two orders of magnitude, with the largest gaps on IC, TSO, and BK. BPCL is not reported for the topology updates because its implementation only supports the weight updates.

Memory during Maintenance. SWPSL keeps no persistent structure for maintenance: its only temporary label sets are released after each batch, and they are negligible for the increases and comparable to the label size for the decreases (Appendix C.3), whereas the PPR of BPCL and WPSL and the position array of M2HL are kept permanently.

6.4 Influence of PLL Number k

Figure 9 evaluates the PLL initialization of the hybrid construction by varying k from 0 (WPSL) to 500. Propagation and final cleanup are unchanged, so the differences between neighboring bars show the redundancy reduction from the additional PLLs. Even a single PLL removes most of the redundancy (86.4% on WT and 84.5% on HO), and the reduction keeps growing with k at a diminishing margin, while the PLLs themselves take longer as k grows, so the construction time is U-shaped in k (detailed analysis in Appendix C.4).

Selection of k. Since the index size and the query time are invariant to k (Section 3.3), the selection only concerns the construction time, whose optimum falls within $[10, 100]$ on 12 of the 13 graphs. Therefore, testing $k \in \{10, 50, 100\}$ on a new graph and keeping the fastest one is sufficient in practice, and even a suboptimal k in this range is still much faster than all the baselines.

6.5 Sensitivity Studies

We further study the sensitivity of SWPSL to the thread number, the update magnitude, and the weight granularity, and the detailed results are reported in Appendix C.5. In short, all the phases of SWPSL scale with the thread number, its maintenance time is robust to the magnitude of the weight changes, and its construction time varies little across the tested weight units on YF, whereas M2HL becomes much slower as the units become finer.

7 RELATED WORK

2-Hop Labeling Methods. Shortest distance is a classic graph problem and had relied on the expensive graph search [10, 21, 27, 28, 56, 77] until the *2-hop labeling* [17] was proposed, which improves the query efficiency by thousands of times but suffers from the slow construction. The existing construction methods fall into three categories. 1) The *search-based* methods: *Pruned Landmark Labeling (PLL)* [7] prunes the searches with the existing labels, and *BPCL* [75], *GLL*, and *PLaNT* [35] run the searches of *PLL* in parallel. 2) The *propagation-based* methods: *Hop Doubling* [32] propagates the labels linearly, *Parallel Shortest Label (PSL)* [33, 43] propagates in parallel by distance, and *WPSL* [68] propagates in parallel by step. 3) The *hierarchy-based* methods: *Hierarchical Hub Labeling* [5, 6] computes the labels with *CH* [27], *IS-Label* [26] builds the hierarchy with the independent sets, and the tree decomposition-based *H2H* [52], *P2H* [15], and *G2H* [38] together with the hierarchical cut-based *HCL* [23] avoid searching entirely when the treewidth is small, with the current fastest *G2H* constructing the labels of the road networks with millions of vertices in seconds using a GPU. Alongside the construction, the maintenance methods have been proposed for each category: *DPLL* [8, 29, 55] maintains *PLL* with graph searching, *WPSL* [68] and *BPCL* [75] maintain their labels with the *PPR*, *M2HL* [65] maintains the propagation-based labels with the distance-based partial reconstruction, and *DH2H* [67, 74, 76] maintains the tree decomposition-based labels along the tree. The hierarchy-based construction and maintenance both rely on the decomposition tree, along which the shortcuts are updated bottom-up and the labels top-down, so their costs grow with the treewidth, which is small on road networks but large on small-world networks [44], and none of them, including *HCL*, has reported results on

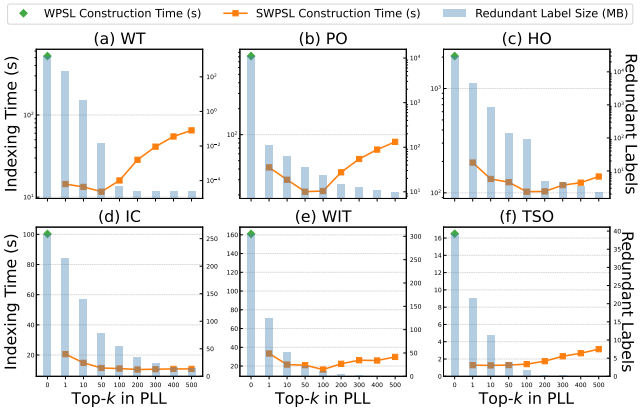


Figure 9: Effect of PLL Number k

such networks. Among the search-based methods, *GLL* cleans the redundancy within each batch of parallel searches like *BPCL*, and *PLaNT* [35] removes the dependency between the roots by labeling a vertex only when the root is the highest-ranked vertex on its shortest path tree, at the cost of exploring far more vertices. Both are static and search from every vertex, whereas *SWPSL* searches only the top-k hubs, propagates the rest in parallel, and maintains the labels with one byte per label. Therefore, *WPSL*, *BPCL*, and *M2HL* are the state-of-the-art for the small-world networks, and they are compared with *SWPSL* in Section 6. Finally, the highway structure [24, 25] accelerates the construction and supports the batch updates on unweighted graphs, but it is not a 2-hop labeling: it answers the queries by the graph search with partial labels, which is three to four orders of magnitude slower than the label intersection (Table 1-(d)), so we only report its query time.

2-Hop Labeling Applications. As the backbone technique, it has improved the efficiency of many complicated path problems from seconds to milliseconds, including but not limited to: *time-dependent path* [19, 39, 41, 42] with the time-dependent edge weights, *constrained shortest path* [45–47, 57, 72] with the multi-dimensional edge weights, *collective spatial keyword path* [37, 40] with multiple stops at different types of POIs, *k nearest neighbour search* [30, 53] in the location-based services, *betweenness centrality* [61], and *shortest path counting* [54, 58, 73]. However, these applications are mainly on road networks and unweighted small-world graphs, but not on weighted small-world graphs, due to the slow construction and the large index size caused by the redundancy issue studied in this work, and *SWPSL* removes this barrier.

8 CONCLUSION

Constructing and maintaining a 2-hop labeling index on weighted small-world networks has been hindered by the redundant labels during construction and the auxiliary structures for maintenance, both of which can be several times larger than the canonical labels themselves. This work analyzes the causes of both theoretically and proposes a space-efficient index SWPSL that relies on neither of them. As a result, the canonical labels are constructed up to two orders of magnitude faster than by the existing methods, and a thousand updates are handled in seconds to tens of seconds with only one byte of auxiliary data per label. In conclusion, this work paves the way for 2-hop labeling to unleash its power in the dynamic weighted small-world networks.

REFERENCES

[1] [n.d.]. Laboratory for Web Algorithmics Datasets. https://law.di.unimi.it/datasets.php.

[2] [n.d.]. Tags Stack Overflow Network. https://www.cs.cornell.edu/~arb/data.

[3] [n.d.]. Weighted Networks Dataset. https://www.kaggle.com/datasets/felipetimbo/weighted-networks.

[4] [n.d.]. Yijun Su's Homepage. https://www.suyijun.tech/dataset/.

[5] Ittai Abraham, Daniel Delling, Andrew V Goldberg, and Renato F Werneck. 2011. A hub-based labeling algorithm for shortest paths in road networks. In *International Symposium on Experimental Algorithms*. Springer, 230–241.

[6] Ittai Abraham, Daniel Delling, Andrew V Goldberg, and Renato F Werneck. 2012. Hierarchical hub labelings for shortest paths. In *European Symposium on Algorithms*. Springer, 24–35.

[7] Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2013. Fast exact shortest-path distance queries on large networks by pruned landmark labeling. In *SIGMOD*. 349–360.

[8] Takuya Akiba, Yoichi Iwata, and Yuichi Yoshida. 2014. Dynamic and historical shortest-path distance queries on large evolving networks by pruned landmark labeling. In *Proceedings of the 23rd international conference on World wide web*. 237–248.

[9] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph based anomaly detection and description: a survey. *Data mining and knowledge discovery* 29, 3 (2015), 626–688.

[10] Reinhard Bauer and Daniel Delling. 2010. SHARC: Fast and robust unidirectional routing. *Journal of Experimental Algorithmics (JEA)* 14 (2010), 2–4.

[11] Michele Bellingeri, D Bevacqua, F Sartori, M Turchetto, F Scotognella, R Alfieri, NKK Nguyen, TT Le, Q Nguyen, and D Cassi. 2023. Considering weights in real social networks: A review. *Frontiers in Physics* 11 (2023), 1152243.

[12] Paolo Boldi, Massimo Santini, and Sebastiano Vigna. 2008. A large time-aware web graph. In *ACM SIGIR Forum*, Vol. 42. ACM New York, NY, USA, 33–38.

[13] Peer Bork, Lars J Jensen, Christian Von Mering, Arun K Ramani, Insuk Lee, and Edward M Marcotte. 2004. Protein interaction networks from yeast to human. *Current opinion in structural biology* 14, 3 (2004), 292–299.

[14] Ulrik Brandes. 2001. A faster algorithm for betweenness centrality. *Journal of mathematical sociology* 25, 2 (2001), 163–177.

[15] Zitong Chen, Ada Wai-Chee Fu, Minhao Jiang, Eric Lo, and Pengfei Zhang. 2021. P2H: efficient distance querying on road networks by projected vertex separators. In *SIGMOD*. 313–325.

[16] Eunjoon Cho, Seth A Myers, and Jure Leskovec. 2011. Friendship and mobility: user movement in location-based social networks. In *Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1082–1090.

[17] Edith Cohen, Eran Halperin, Haim Kaplan, and Uri Zwick. 2003. Reachability and distance queries via 2-hop labels. *SIAM J. Comput.* 32, 5 (2003), 1338–1355.

[18] Sara Cohen, Benny Kimelfeld, and Georgia Koutrika. 2012. A survey on proximity measures for social networks. *Search Computing: Broadening Web Search* (2012), 191–206.

[19] Tangpeng Dan, Xiao Pan, Bolong Zheng, and Xiaofeng Meng. 2023. Double hierarchical labeling shortest distance querying in time-dependent road networks. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2077–2089.

[20] Gianlorenzo D'angelo, Mattia D'emidio, and Daniele Frigioni. 2019. Fully dynamic 2-hop cover labeling. *Journal of Experimental Algorithmics (JEA)* 24 (2019), 1–36.

[21] Edsger W Dijkstra. 1959. A note on two problems in connexion with graphs. *Numerische mathematik* 1, 1 (1959), 269–271.

[22] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC social network benchmark: Interactive workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 619–630.

[23] Muhammad Farhan, Henning Koehler, Robert Ohms, and Qing Wang. 2023. Hierarchical cut labelling-scaling up distance queries on road networks. *Proceedings of the ACM on Management of Data* 1, 4 (2023), 1–25.

[24] Muhammad Farhan, Henning Koehler, and Qing Wang. 2024. BatchHL+: batch dynamic labelling for distance queries on large-scale networks. *The VLDB Journal* 33, 1 (2024), 101–129.

[25] Muhammad Farhan, Qing Wang, Yu Lin, and Brendan McKay. 2022. Fast fully dynamic labelling for distance queries. *The VLDB Journal* 31, 3 (2022), 483–506.

[26] Ada Wai-Chee Fu, Huanhuan Wu, James Cheng, Shumo Chu, and Raymond Chi-Wing Wong. 2013. Is-label: an independent-set based labeling scheme for point-to-point distance querying on large graphs. *Proceedings of the VLDB Endowment* 6, 6 (2013), 457–468.

[27] Robert Geisberger, Peter Sanders, Dominik Schultes, and Daniel Delling. 2008. Contraction hierarchies: Faster and simpler hierarchical routing in road networks. In *Experimental Algorithms: 7th International Workshop, WEA 2008 Provincetown, MA, USA, May 30-June 1, 2008 Proceedings 7*. Springer, 319–333.

[28] Andrew V Goldberg, Haim Kaplan, and Renato F Werneck. 2006. Reach for A*: Efficient point-to-point shortest path algorithms. In *2006 Proceedings of the Eighth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 129–143.

[29] Takanori Hayashi, Takuya Akiba, and Ken-ichi Kawarabayashi. 2016. Fully dynamic shortest-path distance query acceleration on massive networks. In *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*. 1533–1542.

[30] Dan He, Sibo Wang, Xiaofang Zhou, and Reynold Cheng. 2019. An efficient framework for correctness-aware kNN queries on road networks. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 1298–1309.

[31] Sai Hu and Bihai Zhao. 2025. Protein function prediction using GO similarity-based heterogeneous network propagation. *Scientific Reports* 15, 1 (2025), 19131.

[32] Minhao Jiang, Ada Wai-Chee Fu, Raymond Chi-Wing Wong, and Yanyan Xu. 2014. Hop Doubling Label Indexing for Point-to-Point Distance Querying on Scale-Free Networks. *PVLDB* 7, 12 (2014).

[33] Ruoming Jin, Zhen Peng, Wendell Wu, Feodor Dragan, Gagan Agrawal, and Bin Ren. 2020. Parallelizing pruned landmark labeling: dealing with dependencies in graph algorithms. In *Proceedings of the 34th ACM International Conference on Supercomputing*. 1–13.

[34] József Kunegis. 2013. KONECT Network Collection. https://konect.cc.

[35] Kartik Lakhotia, Rajgopal Kannan, Qing Dong, and Viktor Prasanna. 2019. Planting Trees for scalable and efficient Canonical Hub Labeling. *Proceedings of the VLDB Endowment* 13, 4 (2019), 492–505.

[36] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.

[37] Jiajia Li, Qiulin An, Yang Song, Xing Xiong, Lei Li, Fengmei Jin, and Xiaofang Zhou. 2025. Route optimization with collective spatial keywords: A skyline-based approach: J. Li et al. *The VLDB Journal* 34, 5 (2025), 61.

[38] Jiajia Li, Yongzhi Chen, Mengxuan Zhang, and Lei Li. 2025. A CPU-GPU Hybrid Labelling Algorithm for Massive Shortest Distance Queries on Road Networks. *Proceedings of the VLDB Endowment* (2025).

[39] Jiajia Li, Cancan Ni, Dan He, Lei Li, Xiufeng Xia, and Xiaofang Zhou. 2023. Efficient k NN query for moving objects on time-dependent road networks. *The VLDB Journal* 32, 3 (2023), 575–594.

[40] Jiajia Li, Xing Xiong, Lei Li, Dan He, Chuanyu Zong, and Xiaofang Zhou. 2023. Finding top-k optimal routes with collective spatial keywords on road networks. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 368–380.

[41] Lei Li, Sibo Wang, and Xiaofang Zhou. 2019. Time-dependent hop labeling on road network. In *ICDE*. IEEE, 902–913.

[42] Lei Li, Sibo Wang, and Xiaofang Zhou. 2020. Fastest path query answering using time-dependent hop-labeling in road network. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2020), 300–313.

[43] Wentao Li, Miao Qiao, Lu Qin, Ying Zhang, Lijun Chang, and Xuemin Lin. 2019. Scaling distance labeling on small-world networks. In *SIGMOD*. 1060–1077.

[44] Wentao Li, Miao Qiao, Lu Qin, Ying Zhang, Lijun Chang, and Xuemin Lin. 2020. Scaling up distance labeling on graphs with core-periphery properties. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1367–1381.

[45] Ziyi Liu, Lei Li, Mengxuan Zhang, Wen Hua, Pingfu Chao, and Xiaofang Zhou. 2021. Efficient constrained shortest path query answering with forest hop labeling. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1763–1774.

[46] Ziyi Liu, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou. 2022. FHL-cube: Multi-constraint shortest path querying with flexible combination of constraints. *Proceedings of the VLDB Endowment* 15, 11 (2022), 3112–3125.

[47] Ziyi Liu, Lei Li, Mengxuan Zhang, Wen Hua, and Xiaofang Zhou. 2024. Approximate skyline index for constrained shortest pathfinding with theoretical guarantee. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4222–4235.

[48] Ana G Maguitman, Filippo Menczer, Heather Roinestad, and Alessandro Vespignani. 2005. Algorithmic detection of semantic similarity. In *Proceedings of the 14th international conference on World Wide Web*. 107–116.

[49] Jörg Menche, Amitabh Sharma, Maksim Kitsak, Susan Dina Ghiassian, Marc Vidal, Joseph Loscalzo, and Albert-László Barabási. 2015. Uncovering disease-disease relationships through the incomplete interactome. *Science* 347, 6224 (2015), 1257601.

[50] Christian von Mering, Martijn Huynen, Daniel Jaeggi, Steffen Schmidt, Peer Bork, and Berend Snel. 2003. STRING: a database of predicted functional associations between proteins. *Nucleic acids research* 31, 1 (2003), 258–261.

[51] J-P Onnela, Jari Saramäki, Jorkki Hyvönen, György Szabó, David Lazer, Kimmo Kaski, János Kertész, and A-L Barabási. 2007. Structure and tie strengths in mobile communication networks. *Proceedings of the national academy of sciences* 104, 18 (2007), 7332–7336.

[52] Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, and Qing Zhu. 2018. When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks. In *SIGMOD*. 709–724.

[53] Dian Ouyang, Dong Wen, Lu Qin, Lijun Chang, Ying Zhang, and Xuemin Lin. 2020. Progressive top-k nearest neighbors search in large road networks. In

Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data. 1781–1795.

[54] You Peng, Jeffrey Xu Yu, and Sibo Wang. 2023. PSPC: efficient parallel shortest path counting on large-scale graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 896–908.

[55] Yongrui Qin, Quan Z Sheng, Nickolas JG Falkner, Lina Yao, and Simon Parkinson. 2017. Efficient computation of distance labeling for decremental updates in large dynamic graphs. *World Wide Web* 20, 5 (2017), 915–937.

[56] Jagan Sankaranarayanan, Hanan Samet, and Houman Alborzi. 2009. Path oracles for spatial networks. *Proceedings of the VLDB Endowment* 2, 1 (2009), 1210–1221.

[57] Libin Wang and Raymond Chi-Wing Wong. 2023. QHL: A fast algorithm for exact constrained shortest path search on road networks. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–25.

[58] Yiqi Wang, Long Yuan, Zi Chen, Wenjie Zhang, Xuemin Lin, and Qing Liu. 2023. Towards efficient shortest path counting on billion-scale graphs. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. IEEE, 2579–2592.

[59] Duncan J Watts and Steven H Strogatz. 1998. Collective dynamics of 'small-world'networks. *nature* 393, 6684 (1998), 440–442.

[60] Robert West and Jure Leskovec. 2012. Human wayfinding in information networks. In *Proceedings of the 21st international conference on World Wide Web*. 619–628.

[61] Ruizhong Wu, Yehong Xu, Mengxuan Zhang, and Lei Li. 2024. Sub-graph Sharing for Faster Betweenness Centrality Computation on Road Networks. In *Australasian Database Conference*. Springer, 128–142.

[62] Ruizhong Wu, Xinjie Zhou, Mengxuan Zhang, and Lei Li. [n.d.]. A Space-Efficient Distance Labeling for Dynamic Weighted Small-World Networks (Full Version). https://github.com/CSU-WRZ/SWPSL/blob/main/SWPSL_Full.pdf

[63] Dingqi Yang, Bingqing Qu, Jie Yang, and Philippe Cudré-Mauroux. 2020. Lbsn2vec++: Heterogeneous hypergraph embedding for location-based social networks. *IEEE Transactions on Knowledge and Data Engineering* 34, 4 (2020), 1843–1855.

[64] Wei Tang Ji Xiang Yijun Su, Xiang Li and Neng Gao. 2018. Next Check-in Location Prediction via Footprints and Friendship on Location-Based Social Networks. In *IEEE International Conference on Mobile Data Management, MDM 2018*. 251–256. https://doi.org/10.1109/MDM.2018.00044

[65] Yuanyuan Zeng, Yixiang Fang, Kun Chen, Yangfan Li, and Chenhao Ma. 2025. Efficient Maintenance of 2-Hop Labeling Index on Dynamic Small-World Graphs. *Proceedings of the VLDB Endowment* 18, 7 (2025), 2005–2017.

[66] Yuanyuan Zeng, Chenhao Ma, and Yixiang Fang. 2024. Distributed Shortest Distance Labeling on Large-Scale Graphs. *Proceedings of the VLDB Endowment* 17, 10 (2024), 2641–2653.

[67] Mengxuan Zhang, Lei Li, Wen Hua, Rui Mao, Pingfu Chao, and Xiaofang Zhou. 2021. Dynamic hub labeling for road networks. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 336–347.

[68] Mengxuan Zhang, Lei Li, Wen Hua, and Xiaofang Zhou. 2021. Efficient 2-hop labeling maintenance in dynamic small-world networks. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 133–144.

[69] Mengxuan Zhang, Lei Li, Goce Trajcevski, Andreas Zufle, and Xiaofang Zhou. 2023. Parallel Hub Labeling Maintenance With High Efficiency in Dynamic Small-World Networks. *TKDE* (2023).

[70] Mengxuan Zhang, Lei Li, and Xiaofang Zhou. 2021. An experimental evaluation and guideline for path finding in weighted dynamic network. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2127–2140.

[71] Mengxuan Zhang, Xinjie Zhou, Lei Li, and Xiaofang Zhou. 2025. Partitioned Dynamic Hub Labeling for Large Road Networks. *IEEE Transactions on Knowledge and Data Engineering* (2025).

[72] U Zhang, Long Yuan, Wentao Li, Lu Qin, and Ying Zhang. 2021. Efficient label-constrained shortest path queries on road networks: A tree decomposition approach. *Proceedings of the VLDB Endowment* (2021).

[73] Yikai Zhang and Jeffrey Xu Yu. 2020. Hub labeling for shortest path counting. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1813–1828.

[74] Yikai Zhang and Jeffrey Xu Yu. 2022. Relative subboundedness of contraction hierarchy and hierarchical 2-hop index in dynamic road networks. In *SIGMOD*. 1992–2005.

[75] Xinjie Zhou, Mengxuan Zhang, Lei Li, and Xiaofang Zhou. 2024. Scalable distance labeling maintenance and construction for dynamic small-world networks. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4573–4585.

[76] Xinjie Zhou, Mengxuan Zhang, Lei Li, and Xiaofang Zhou. 2025. High throughput shortest distance query processing on large dynamic road networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1166–1179.

[77] Andy Diwen Zhu, Hui Ma, Xiaokui Xiao, Siqiang Luo, Youze Tang, and Shuigeng Zhou. 2013. Shortest path and distance queries on road networks: towards bridging theory and practice. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*. 857–868.

Algorithm 6: Pivot-Triggered One-Pass Removal (SOP-RLR)

Input: Graph $G(V, E)$, Current Step h, Current Labels $L_{\leq h}$
Output: Updated labeling with pruned redundancy $L_{\leq h}$

1 $r_{h-1} \leftarrow \max\{r(w) \mid (w, \cdot) \in L_{h-1}\}$, $r_h \leftarrow \max\{r(w) \mid (w, \cdot) \in L_h\}$ ($-\infty$ if $L_h = \emptyset$);
2 $W_h \leftarrow \{w \in V \mid r_h < r(w) \leq r_{h-1}\}$ ▷ Hubs finished after step h;
3 **if** $W_h = \emptyset$ **then return** $L_{\leq h}$;
4 $D[v] \leftarrow \emptyset,\ \forall v \in V$;
5 **foreach** $w \in W_h$ **do**
6 **foreach** $v \in V$ *with* $w \in C(v)$ ***in parallel*** **do**
7 **foreach** $(u, d) \in L[v]$ *with* $r(u) < r(w)$ ***and*** $w \in C(u)$ **do**
8 **if** $L[v][w] + L[u][w] \leq d$ **then**
9 $D[v].insert(u)$ ▷ Mark the entry for deletion
10 $L[v] \leftarrow \{(u, d) \in L[v] \mid u \notin D[v]\},\ \forall v \in V$;
11 **return** $L_{\leq h}$

A PROOFS

A.1 Proof of Theorem 1

Case 1: $L(u)[w] + L(v)[w] < L(v)[u]$. In this case, the common hub w provides a shorter connection between u and v than that represented by $L(v)[u]$. Hence, $L(v)[u]$ cannot contribute to any shortest distance query and is therefore redundant. **Case 2:** $L(u)[w] + L(v)[w] = L(v)[u]$. Since w lies on a path between u and v and has a higher order than u, vertex u cannot be the highest-order vertex among the vertices on that path. Consequently, $L(v)[u]$ must violate the canonical labeling constraint stated in Constraint 2, which requires u to be the highest-order vertex on every shortest path between u and v. Note that $r(w) > r(u)$ already implies $w \neq u$, and the condition holds as long as one such w exists, no matter how many shortest paths there are. Therefore, $(u, L(v)[u])$ is non-canonical, and thus redundant. Since $(u, L(v)[u])$ is redundant in both cases, the theorem follows.

A.2 Proof of Theorem 2

By Theorem 1, if there exists a vertex w satisfying $r(w) > r(u)$, $(w, L(u)[w]) \in L(u)$, $(w, L(v)[w]) \in L(v)$, and $L(u)[w]+L(v)[w] \leq L(v)[u]$, then the label $(u, L(v)[u]) \in L(v)$ is redundant. It remains to show that once every vertex with rank higher than $r(v)$ has finished propagation, the absence of such a vertex implies that $(u, L(v)[u])$ is non-redundant. Assume, for contradiction, that all vertices w satisfying $r(w) > r(v)$ have completed propagation, no vertex satisfies the condition of Theorem 1, yet $(u, L(v)[u])$ is still redundant.

We consider the two possible types of redundant labels. **Type-2 redundancy:** $L(v)[u] > dist(u, v)$. Since all higher-ranked vertices have completed propagation and the WPSL[68] index satisfies the 2-hop cover property, the query operation over $L(u)$ and $L(v)$ returns the exact shortest-path distance: $Q(L(u), L(v)) = dist(u, v)$. Therefore, $Q(L(u), L(v)) < L(v)[u]$. Let w be the common hub that realizes the query result. Then $L(u)[w] + L(v)[w] < L(v)[u]$. Thus, w satisfies the condition of Theorem 1, implying that $(u, L(v)[u])$ can be identified as redundant, which contradicts the assumption that no such vertex exists.

Type-3 redundancy: $L(v)[u] = dist(u, v)$, but $(u, L(v)[u])$ is non-canonical. Then there exists a vertex w on a shortest path between u and v such that $r(w) > r(u)$. Let w' be the highest-ranked vertex among all vertices appearing on any shortest path between u and v. Since the WPSL construction preserves all canonical labels, the entries $(w', L(u)[w']) \in L(u)$ and $(w', L(v)[w']) \in L(v)$ must both exist after propagation completes. Moreover, $L(u)[w'] + L(v)[w'] = dist(u, v) = L(v)[u]$. Hence, w' satisfies all conditions of Theorem 1. Therefore, $(u, L(v)[u])$ can be identified as redundant, again contradicting the assumption that no such vertex exists.

Both cases lead to contradictions. Therefore, after all vertices with rank higher than $r(v)$ have completed propagation, if no vertex satisfies the condition of Theorem 1, then $(u, L(v)[u])$ cannot be redundant. Equivalently, it must be canonical. Hence, the redundancy of labels in $L(v)$ can be completely verified once every vertex w satisfying $r(w) > r(v)$ has finished propagation.

A.3 Proof of Lemma 1

A label with hub w in L_h can only be generated from a label with hub w in L_{h-1} (line 5 of Algorithm 1). Hence, if no label of w is generated in L_{h-1}, no label of w is generated in L_h either, and by induction in any later step. The set of propagating hubs therefore shrinks monotonically, and so does its maximum rank r_h.

A.4 Canonicity of the Pivot-Triggered Removal (SOP-RLR)

Algorithm 6 gives the pivot-triggered removal described in Section 3.2; its check is in line 8. The rank intervals $(r_h, r_{h-1}]$ partition the ranks, so every hub is processed as a pivot in exactly one batch W_h, and by Lemma 1 its labels are final at that time. Consider a redundant label $(u, d) \in L(v)$. By Theorem 2, once all the vertices ranked above $r(v)$ have finished propagating, some hub w with $r(w) > r(u)$ satisfies the condition of Theorem 1 for (u, d), and w is processed in the batch of the step at which it finishes. If (u, d) already exists when w is processed, line 8 of Algorithm 6 marks it. If (u, d) is generated at a later step, the pre-insertion query $Q(L_{<h}[u], L_{<h}[v])$ of Algorithm 1 already returns a distance no larger than d through w, so the entry is never inserted. Hence, no Type-2 or Type-3 redundant label survives, while the Type-1 redundancy is removed by the insertion semantics. Conversely, every removal is certified by Theorem 1, and the witnesses of the removed labels are canonical labels that are never removed, so the 2-hop cover is preserved. Therefore, SOP-RLR outputs the canonical labeling.

A.5 Proof of Theorem 3

1) The PLL searches run from the top-k vertices in decreasing rank order, so they produce exactly the canonical labels whose hubs are among the top-k [7]. 2) The propagation generates every canonical label $(w, dist(w, v))$ with w outside the top-k: along a shortest path from w to v with the fewest hops, w is the highest-ranked vertex of every prefix, so by induction the candidate of each prefix is proposed and accepted unless it already exists, because a pre-insertion answer no larger than its distance would place a hub ranked above w on a shortest path. Hence, the converged labeling contains all the canonical labels and satisfies the 2-hop cover. 3) In the final scan, every hub has finished, so by Theorem 2 each non-canonical entry has a witness and is removed, whereas a canonical entry has none and survives. Since the witnesses are canonical labels that survive, the 2-hop cover is preserved and the output is exactly the canonical labeling.

A.6 Correctness of the Parallel Construction and Maintenance

The parallel propagation phases partition their output by vertex. In SWPSL, each propagation step reads only the finalized label sets L_{h-1} and $L_{<h}$, and each vertex writes to its own output set $L_h[v]$. Therefore, no concurrent read-write conflicts occur between threads, ensuring that parallel execution produces the same stepwise labels as sequential execution. The top-k PLL labels are canonical and remain available to every pruning query. Propagation preserves the 2-hop cover and the canonical entries. After convergence, SWPSL's final cleanup reads a fixed snapshot and writes survivors by vertex to a separate output. An entry is removed only by a shorter answer or an equal answer through a different pivot. Canonical entries cannot satisfy either condition, and their surviving witnesses certify all other removals. Thus, the parallel cleanup preserves the 2-hop cover and produces the canonical labeling. The direct SOP-RLR scheme is separate from this construction. Its pivot checks read a fixed snapshot within each batch and apply the deletion marks only after the batch.

In Algorithm 3, L, $L^{\downarrow}$, and $A^{\downarrow} = \{v \mid L^{\downarrow}[v] \neq \emptyset\}$ are fixed before filtering. Each phase reads these inputs and writes only to the output $L'[v]$ of its assigned vertex. The phases execute in sequence. No query reads an output being modified by another thread. Algorithm 2 shows LCP in push form. An equivalent parallel pull form assigns each target vertex to one thread, which gathers candidates from its neighbors' current frontiers, keeps the minimum candidate per hub, and publishes only strict improvements over the accumulated $L^{\downarrow}$ at the end of the round. This produces the same next frontier as the push form.

In Algorithm 4, the edge phase reads the original snapshot L_G, and the deletion requests of the increased edges that share an endpoint are collected independently and combined by set union at the synchronization point. During step-h validation, each vertex produces its own requests, which are published after that step. Predecessor checks read the deletion state committed by earlier phases, so marked predecessors cannot provide support. Physical deletion from L_G is deferred until detection finishes. Self labels are never marked. In Algorithm 5, the labeling L is read-only within a round, each thread only writes to its own $L_{(t)}[v]$, and $L_{(t)}$ is merged into L only after the round (line 11), which avoids any read-write conflict. Therefore, the parallel execution of all the algorithms generates exactly the same labels as the sequential one.

A.7 Proof of Theorem 4

We prove by contradiction. Assume there exists a vertex pair (u, v) such that after edge-weight decreases, the shortest distance in the updated graph $G^{\downarrow}$ satisfies $d_{G^{\downarrow}}(u, v) < Q(L[u] \cup L^{\downarrow}[u], L[v] \cup L^{\downarrow}[v])$ and the 2-hop cover constraint is violated. Let P be a shortest path from u to v in $G^{\downarrow}$. We consider two cases: **Case 1:** P contains no edge with decreased weight. Then $d_{G^{\downarrow}}(u, v) = d_G(u, v)$, and since the original label L_G satisfies the 2-hop cover constraint, we have $Q(L[u], L[v]) = d_G(u, v)$, which contradicts our assumption. **Case 2:** P contains at least one edge $e(a, b)$ whose weight has decreased. Since $\phi^{\downarrow}(a, b) < Q(L_G[a], L_G[b])$, the edge (a, b) is added to the active set $\mathcal{E}$ (lines 2-5). WCP phase ensures that any vertex whose distance to b (or a) becomes shorter through the decreased edge is captured in $L^{\downarrow}$. Furthermore, the LCP phase iteratively propagates these new labels to their neighbors. ensuring that all distance decreases are fully propagated throughout the graph. By induction on the propagation rounds, any vertex pair whose shortest distance decreases will eventually have a corresponding label in $L \cup L^{\downarrow}$ that correctly answers their distance via some hub. This contradicts our initial assumption that (u, v) violates the 2-hop cover constraint. Therefore, $L \cup L^{\downarrow}$ satisfies 2-hop cover constraint. The strict-improvement check against $L^{\downarrow}$ does not suppress a necessary propagation: a rejected candidate has an equal or shorter entry for the same hub already accumulated, whose propagation is either pending or has already occurred. Extending that entry over any edge yields a distance no larger than extending the rejected candidate. Thus omitting equal or worse candidates from the next frontier preserves the argument above.

A.8 Proof of Theorem 5

Let $M = L \cup L^{\downarrow}$, keeping the minimum distance for each hub. By Theorem 4, M answers every distance exactly. Its entries are upper bounds on the updated distances and respect the hub rank order. Consequently, M contains every canonical entry for this order: for a pair (u, v) for which u is highest-ranked over all shortest u–v paths, a covering pivot has rank at least $r(u)$, and a higher-ranked pivot would lie on a shortest u–v path, contradicting the canonical property. Hence the covering pivot must be u, and the exact entry is present at v.

No canonical entry (u, d) can satisfy the redundancy predicate. A query cannot return less than the true distance d. An equal answer through a different pivot would place a higher-ranked vertex on a shortest u–v path. Thus all canonical entries survive. Conversely, an overestimated entry has $q_g < d$, and a dominated entry has $q_g = d$ with an alternative higher-ranked pivot. The canonical entries witnessing these answers also survive, so simultaneous filtering does not introduce a circular dependency or lose coverage. In particular, an equal answer through u alone is not a deletion certificate, whereas a shorter new entry with hub u correctly eliminates its old longer copy.

The four local queries enumerate all old/new entry combinations, so their predicates have the same disjunction as the global redundancy test. For a new entry, the pre-insertion test gives $q_1 > d$. For an old entry, the original canonical labeling gives $q_1 = d$ and $P_1 = \{u\}$. Thus R_1 is false in both phases. Since $A^{\downarrow}$ is fixed from the unfiltered input $L^{\downarrow}$, every skipped local query has an empty input side and returns $(+\infty, \emptyset)$. The four cases therefore omit no possible deletion certificate, and Case 1 explicitly copies its old entry to L'. The output contains exactly the canonical entries and satisfies the 2-hop cover constraint.

A.9 Proof of Lemma 2

By the step invariant (Section 3.3), a label $(u, d) \in L_1[v]$ of step 1 is the edge label of $e(u, v)$ with $d = \phi_G(u, v) = dist_G(u, v)$. If $e(u, v) \notin E^{\uparrow}$, the edge still has length d in $G^{\uparrow}$, so $dist_{G^{\uparrow}}(u, v) \leq d$ and the label remains valid. Hence, only the edge labels of the increased edges can become invalid among the labels of step 1. Such an edge label $(b, \phi_G(a, b)) \in L_1[a]$ is invalid if and only if $dist_{G^{\uparrow}}(a, b) > \phi_G(a, b)$, *i.e.,* no other path of length $\phi_G(a, b)$ connects a and b. Algorithm 4 removes it in both cases, because checking the condition would

require a shortest path search per edge, and a valid edge label that survived with step 1 would no longer be an edge label, which breaks the step invariant for the later batches.

A.10 Proof of Lemma 3

Each label $(w, d) \in L_h[v]$ is generated from a predecessor $(w, d') \in L_{h-1}[u]$ via some neighbor $u \in N(v)$, where $d = \phi(v, u) + d'$. An increased connecting edge or an invalid predecessor can destroy this particular support. This alone does not imply that (w, d) is invalid, because another predecessor may still certify a path of length d. Conversely, if (w, d) is supported by at least one edge $e(v, u) \notin E^{\uparrow}$ and a valid predecessor $(w, d') \in L_{h-1}[u]$, then the path $\langle v, u, \ldots, w\rangle$ still has length d in $G^{\uparrow}$, so $d \geq dist_{G^{\uparrow}}(v, w)$ and (w, d) cannot underestimate the updated distance, *i.e.,* it is valid.

A.11 Proof of Theorem 6

All step-0 self labels remain valid and are retained. We prove removal completeness by induction on the positive generation steps. A label whose hub is in $D[v]$ is treated as absent by *CheckPredecessor*, and all marks from earlier phases are visible before the next step begins. **[Base]:** *All invalid labels at step $h = 1$ are removed.* Suppose $(u, d) \in L_{G,h=1}[v]$ is an invalid label. Since it has step 1, it is an edge label (Section 5), so $d = dist_G(u, v) = \phi_G(u, v)$, meaning that the shortest path $sp_G(u, v)$ is the edge $e(u, v)$. If (u, d) is invalid, then $dist_{G^{\uparrow}}(u, v) > dist_G(u, v) = \phi_G(u, v)$, so $e(u, v)$ has increased (Lemma 2), and the edge phase of Algorithm 4 marks (u, d) at v while retaining the self label $(u, 0)$ at u. Hence every invalid step-1 entry is marked without deleting a self label. **[Induction]:** *All invalid labels at step $h > 1$ are removed, provided that all invalid labels at step $h - 1$ have been removed.* Let $(w, d) \in L_{G,h}[v]$ be invalid, *i.e.,* $d < dist_{G^{\uparrow}}(v, w)$, and consider any neighbor $x \in N(v)$ examined by *CheckPredecessor*, *i.e.,* $e(v, x) \notin E^{\uparrow}$, with a predecessor $(w, d') \in L_{G,h-1}[x]$ such that $d' = d - \phi(v, x)$. Since $\phi^{\uparrow}(v, x) = \phi(v, x)$, the triangle inequality gives $dist_{G^{\uparrow}}(x, w) \geq dist_{G^{\uparrow}}(v, w) - \phi(v, x) > d - \phi(v, x) = d'$, so the predecessor (w, d') is itself invalid and has been marked by the induction hypothesis, hence $w \in D[x]$. The neighbors with $e(v, x) \in E^{\uparrow}$ are skipped by *CheckPredecessor*. Hence, (w, d) has no valid predecessor left and is marked at step h. Conversely, a label that keeps a valid predecessor is valid by Lemma 3, so the removal never stops before all the invalid labels are found, and Algorithm 4 removes all of them.

Therefore, the completeness of Algorithm 4 relies only on the base step: once the edge labels of the increased edges are removed, every invalid label of a higher step is found by the predecessor validation, which is why the edge phase needs neither a distance check nor the pivots between the endpoints.

Remark. Lemma 3 gives a sufficient, but not necessary, condition for retaining a label. Thus, *CheckPredecessor* may conservatively remove a valid label when all its recorded predecessor supports from step $h - 1$ have been invalidated or removed, even if an equal-length path remains through a label generated at a different step, and the edge phase likewise removes the edge label of an increased edge even when another path of the same length keeps it valid. This removal is safe: Theorem 6 guarantees that no invalid label survives, and the recovery phase in Section 5.2 restores the 2-hop cover by generating any labels needed for coverage. A removed label need not be reinserted if the remaining labels already cover the corresponding vertex pair.

A.12 Proof of Pruning Condition 1

If neither v nor w belongs to $\mathcal{A}^{\uparrow}$, then neither of them loses any label in the deletion phase (Algorithm 4). Hence, their original label sets $L_G[v]$ and $L_G[w]$ remain valid and can answer $Q(L_G[v], L_G[w]) = dist_G(v, w)$ correctly. Since the increased edge cannot shorten any shortest path, we have $dist_G(v, w) \leq dist_{G^{\uparrow}}(v, w)$. Therefore, the existing valid labels already provide the correct distance in $G^{\uparrow}$, that is, $Q(L[v], L[w]) = dist_{G^{\uparrow}}(v, w)$. Then inserting a label (w, d) into $L(v)$ would be redundant, as $dist(v, w)$ is already covered by the existing hubs. Consequently, such vertex pairs can be safely pruned during propagation.

A.13 Proof of Pruning Condition 2

Since $w \notin \mathcal{A}^{\uparrow}$, the label set $L_G[w]$ remains unchanged and thus valid in $G^{\uparrow}$. The pivot vertex x that witnesses a shortest distance between two vertices (*i.e.,* $Q(L_G[u], L_G[v])$) satisfies $r(x) \geq \max\{r(u), r(v)\}$. If the maximum deleted rank at v is $Max_r(v) < r(w)$, then every pivot x with $r(x) > r(w)$ in $L_G[v]$ survives the deletion phase. Consequently, the distance query using the *remaining* labels at v and the unchanged labels at w still attains the updated distance: $Q(L[v], L[w]) = dist_{G^{\uparrow}}(v, w)$. Therefore, exactly as in Pruning Condition 1, the candidate (w, d) is already covered by the existing hubs and can be safely skipped without further testing.

A.14 Proof of Theorem 7

If the pruning conditions (line 7) are disabled, Algorithm 5 is equivalent to the standard WPSL construction, which has been proven to guarantee complete 2-hop coverage, so the unpruned version is correct. The two pruning conditions only exclude label pairs that are already correctly covered by existing valid hubs, so they merely prune the redundant candidates, without affecting the correctness. Hence, the resulting labeling $L_{\text{old}} \cup L_{\text{new}}$ preserves the 2-hop cover property and remains fully correct under $G^{\uparrow}$.

A.15 Proof of Theorem 8

As all the invalid labels are deleted from the original canonical G's label set G, the remaining labels L_{old} are still valid and canonical in G. Hence, it cannot contain *Type-1* or *Type-2* redundancy. The only possible redundancy would be *Type-3*, where a label $(u, d) \in L_{old}[v]$ could be dominated by another vertex x if $r(x) > r(v)$, $r(x) > r(u)$, and $dist_{G^{\uparrow}}(v, x) + dist_{G^{\uparrow}}(x, u) = dist_{G^{\uparrow}}(v, u) = d = \text{dist}_G(v, u)$. However, since $dist_{G^{\uparrow}}(v, x) \geq dist_G(v, x)$ and $dist_{G^{\uparrow}}(x, u) \geq dist_G(x, u)$, such a vertex x must have also dominated (u, d) in the original graph G. Therefore, (u, d) would never appear in $L_G[v]$ under a canonical labeling scheme. This contradiction proves that no label in L_{old} can become dominated under $G^{\uparrow}$. Hence, L_{old} is free of redundancy.

B COMPLEXITY ANALYSIS

We follow the notation of Section 3.3, where $n = |V|$, $m = |E|$, p is the number of threads, δ is the maximum label size of a vertex, and a distance query takes $O(\delta)$ time.

B.1 Index Construction (Section 3.3)

Propagation depth versus topological diameter. The unweighted hop distance minimizes the number of edges, whereas a weighted shortest path minimizes the total edge weight. These quantities need not coincide: consider a complete graph on $n \geq 3$ vertices, with weight 1 on the edges of a spanning path and weight n on all other edges. Its unweighted diameter is 1, but the weighted shortest path between the spanning path's endpoints contains $n-1$ edges. Thus, small topological diameter alone does not guarantee few-hop weighted shortest paths. We use the actual number of propagation rounds H in the analysis below, rather than identifying H with the unweighted hop diameter.

Hybrid construction. For sequential PLL initialization, the k graph searches cost $O(k(m + n \log n))$. The i-th search performs at most n pruning queries over at most $O(i)$ existing hubs per label set. These queries add $O(nk^2)$ work across all searches. In each propagation round, a vertex v collects $O(D(v)\delta)$ candidates and checks each with an $O(\delta)$-time query, giving $O(Hm\delta^2)$ total work. After propagation converges, the final cleanup scans at most $O(n\delta)$ entries in a fixed labeling. Each entry requires one query returning the minimum distance and all minimizing pivots, which takes $O(\delta)$ time. Thus, this final scan costs $O(n\delta^2)$ work. This bound does not rely on the per-pivot one-pass property of SOP-RLR. The total work is $O(k(m+n\log n)+nk^2+Hm\delta^2+n\delta^2)$, with $O(n\delta)$ space for labels and temporary outputs. Under ideal load balance and ignoring synchronization overhead, distributing propagation and cleanup over p threads gives $O(k(m + n\log n) + nk^2 + (Hm\delta^2 + n\delta^2)/p)$ time.

Direct SOP-RLR. Let δ_S be the maximum label size during SOP-RLR's execution. Each pivot is processed once, but scans all n vertices and up to δ_S entries at each eligible vertex. With constant-time hub lookups, the pruning loops of Algorithm 6 therefore require $O(n^2\delta_S)$ work in the worst case. A label can be checked against multiple pivots. This pruning cost is not incurred by the hybrid construction.

B.2 Decrease Maintenance (Section 4)

Let H_d be the number of LCP rounds and m_d be the maximum sum of degrees of the vertices that receive new labels in one round. In Algorithm 2, the initial detection takes one query per decreased edge, *i.e.,* $O(|E^{\downarrow}|\delta)$ time. WCP checks $O(\delta)$ candidate hubs for each active edge with one query each, so it takes $O(|E^{\downarrow}|\delta^2)$ time, and LCP takes $O(H_d m_d \delta^2)$ time, since each vertex with new labels propagates $O(\delta)$ labels to each of its neighbors with one query per candidate. Therefore, Algorithm 2 takes $O(|E^{\downarrow}|\delta^2 + H_d m_d \delta^2)$ time in total. In Algorithm 3, the new labels, at most $O(H_d m_d \delta)$ in total, are each checked with three queries in $O(|L^{\downarrow}|\delta)$ time, while the old labels are only checked for the pairs with an affected endpoint, resulting in a worst-case bound of $O(n\delta^2)$, as pairs with no affected endpoints (Case 1) are skipped without any query. Both algorithms are parallelized over vertices with p threads (Section 4.2), and the extra space is the temporary new label set $L^{\downarrow}$, which is released after the update.

B.3 Increase Maintenance (Section 5)

The edge phase of Algorithm 4 removes the edge label of each increased edge with a constant-time lookup per edge, so it takes $O(|E^{\uparrow}|)$ time. The predecessor validation checks each label against the labels of its neighbors, so each step takes $O(\sum_v D(v)\delta) = O(m\delta)$ time and all the steps take $O(Hm\delta)$ time in total, where $H = maxStep$ is the maximum recorded generation step in the input index. Let H_i be the number of recovery rounds and m_i be the maximum sum of degrees of the vertices that receive new labels in one round. The pruned propagation in Algorithm 5 takes $O(H_i m_i \delta^2)$ time, since each surviving candidate incurs one query, while the two pruning conditions skip the candidates of the unaffected pairs without any query. The redundancy removal on the recovered labels takes $O(|L_{new}|\delta)$ time with $|L_{new}| = O(H_i m_i \delta)$. The edge phase is parallelized over the updated edges, while predecessor validation, recovery, and redundancy filtering are parallelized over vertices with p threads, and the extra space is the one-byte generation step of each label together with the temporary L_{new}, which is released after the update. Compared with the PPR-based maintenance, SWPSL trades the dependency lookups of the PPR for the predecessor validation and the pruned propagation, both of which are bounded by the labels of the affected vertices and require no persistent structure.

C TOPOLOGY UPDATES

The edge and vertex insertions are reduced to the weight decrease in Section 4, and the edge and vertex deletions are reduced to the weight increase in Section 5, as detailed below.

C.1 Insertion Update Maintenance

We next discuss how to support *edge* and *vertex insertion* in SWPSL. Both operations can be reduced to the *edge decrease*, so they can be handled by the same mechanisms introduced in Alg. 2 and 3.

Edge insertion. Inserting a new edge $e(a, b)$ with weight w into graph G can be viewed as a special case of edge weight decrease. Specifically, we treat the original graph G as containing $e(a, b)$ with an *infinite* weight, which is then decreased to w. Under this formulation, the edge insertion is equivalent to adding $e(a, b)$ into the decreased edge set $E^{\downarrow}$. As a result, we can directly invoke the weight-decrease update procedures, including the new label propagation and the redundant label removal.

Vertex insertion. Inserting an isolated vertex alone does not affect the shortest distances and has no impact on the labels. We therefore consider the vertex insertion together with the insertion of its incident edges. When a new vertex v is inserted, we first initialize its label set as $L(v) = \{(v, 0)\}$. To preserve the canonical labeling order, the new vertex is assigned the new lowest rank. After that, each newly inserted incident edge $e(a, b)$ is processed as an edge insertion, and hence reduced to the edge weight decrease case described above.

C.2 Deletion Update Maintenance

This section discusses how to support *edge* and *vertex deletion*. Similar to the insertion, both operations can be reduced to the edge *increase* and handled by the increase update methods. **Edge deletion:** Deleting an edge $e(a, b)$ can be equivalently modeled as increasing its weight to infinity, so the removal of $e(a, b)$ is treated as an edge weight increase update. Therefore, we insert $e(a, b)$ into

Table 2: Construction time (s) under different distance units on BK and YF.

Dataset	Algorithm	Distance unit		
		10^4 m	10^3 m	10^2 m
BK	M2HL	10.18	107.02	1130
	SWPSL	**0.71**	**0.94**	**1.02**
YF	M2HL	1.77	14.81	130.34
	SWPSL	**0.15**	**0.17**	**0.27**

the increased edge set $E^{\uparrow}$ and directly apply the procedures for the weight increase, including the invalid label detection (Algorithm 4) and the subsequent label recovery (Algorithm 5). **Vertex deletion:** Deleting a vertex v requires deleting all the edges incident to v, so the vertex deletion is reduced to a set of edge deletions.

C.3 Memory during Maintenance

During maintenance, SWPSL uses temporary label sets, including the new label set $L^{\downarrow}$ for decreases (Algorithm 2) and the recovered label set L_{new} for increases (Algorithm 5), and these sets are released after the corresponding batch is processed. We report the three largest temporary label footprints observed during maintenance. For 1k decreases, the peak sizes of the temporary structures are 164.51MB, 344.59MB, and 707.58MB on BK, GO, and IC, respectively, which are comparable to the sizes of their canonical labels (Table 1-(c)), and for 1k increases, the recovered labels only take 3.15MB, 2.56MB, and 1.21MB. In contrast, the PPR of BPCL and WPSL has to be kept permanently and grows with the updates, and the position array of M2HL has to be kept for every label.

C.4 Influence of PLL Number k

This subsection gives the detailed analysis of Figure 9, which is summarized in Section 6.4.

As shown in Figure 9, we study the influence of the PLL number k in the hybrid construction on the indexing time and the size of the redundant label entries by varying it from 0 (WPSL), 1 to 500. The difference between the neighboring bars shows how many redundant labels can be pruned by the additional PLLs. First, the benefit of PLLs on the redundancy reduction is dramatic and immediate. For some networks, even a single PLL (k=1) drastically cuts the redundancy by 86.4% for WT (from 1,586MB to 216MB) and 84.5% for HO (from 30,037MB to 4,662MB). This redundancy reduction continues to improve with a larger k, but the marginal benefit diminishes after 200. Meanwhile, as PLL is slower than WPSL, the more PLLs we run, the longer it takes. Therefore, the construction time exhibits a clear U-shaped pattern with the optimal k falling around 1-50 for the smaller graphs and 100-200 for the larger graphs. As k further increases, the initial PLL cost begins to dominate, causing a gradual time increase. When k grows from 100 to 500, the construction time increases and slowly surpasses that of k=1.

Selection of k. Since the index size and the query time are invariant to k (Section 3.3), the selection only concerns the construction time. The experiments show that the optimum falls within $[10, 100]$ on 12 of the 13 graphs, and even on IC, where the optimum lies outside this range, the best k in $\{10, 50, 100\}$ is only 0.65s slower than the optimum, which is negligible compared with the gap to the baselines. Therefore, the rule of testing $\{10, 50, 100\}$ on a new graph and keeping the fastest one is sufficient in practice. Even a suboptimal k in this range is still much faster than all the baselines.

C.5 Sensitivity Studies

This subsection reports the detailed results of the sensitivity studies summarized in Section 6.5.

Thread Scalability. Figure 10 shows the construction time and the maintenance time of 1k updates under 8, 16, 32, and 64 threads on SK, WI, HO, BK, GO, and IC. All the phases of SWPSL scale with the thread number: increasing the thread count from 8 to 64 reduces construction, decrease maintenance, and increase maintenance times by factors of up to 5.04×, 5.00×, and 4.29×, respectively, all on IC. The scaling is sublinear and varies by workload. These consistent runtime reductions show that additional threads benefit SWPSL across both construction and maintenance workloads.

Update Magnitude. Figure 11 presents the maintenance time for the same batches of 1,000 edge weight updates while varying the multiplicative weight-scaling factor to study the impact of the change magnitude. For decreases, a smaller scaling factor represents a larger weight reduction, which may shorten more shortest paths and trigger additional label generation. The observed growth in maintenance time remains moderate: for example, reducing the factor from 0.8 to 0.2 increases the time from 3.25 s to 5.77 s on BK and from 16.55 s to 27.93 s on IC. For increases, maintenance time changes only slightly across the tested factors, ranging from 1.14 s to 1.28 s on GO and from 5.95 s to 6.10 s on IC. Overall, these results indicate that SWPSL maintains stable update performance across the evaluated weight-change magnitudes, with moderate sensitivity to decreases and limited sensitivity to increases.

Weight Granularity. To further examine the sensitivity to the numeric distance scale discussed in Section 1, Table 2 reports the construction time of M2HL and SWPSL on BK and YF under three different distance units, ranging from 10^4 m to 10^2 m. As the distance unit becomes finer, the construction time of M2HL increases dramatically on both datasets. Specifically, it increases from 10.18s to 1,130s on BK and from 1.77s to 130.34s on YF. In contrast, the construction time of SWPSL remains relatively stable on both datasets. These results indicate that SWPSL is substantially less sensitive to the granularity of distance weights than M2HL. Although M2HL is relatively efficient under coarse distance units, its construction cost grows rapidly as the distance representation becomes more fine-grained. This observation further demonstrates the robustness and scalability of SWPSL under different distance scales.

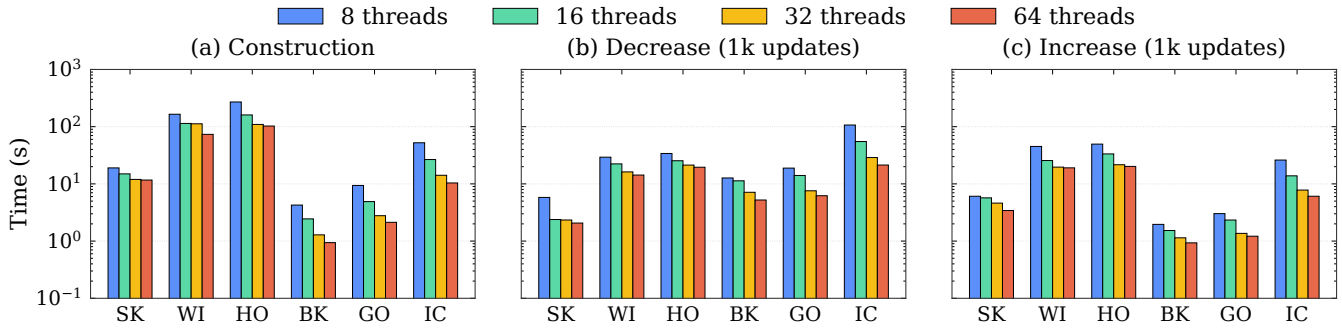


Figure 10: SWPSL Construction and Maintenance under Different Thread Counts

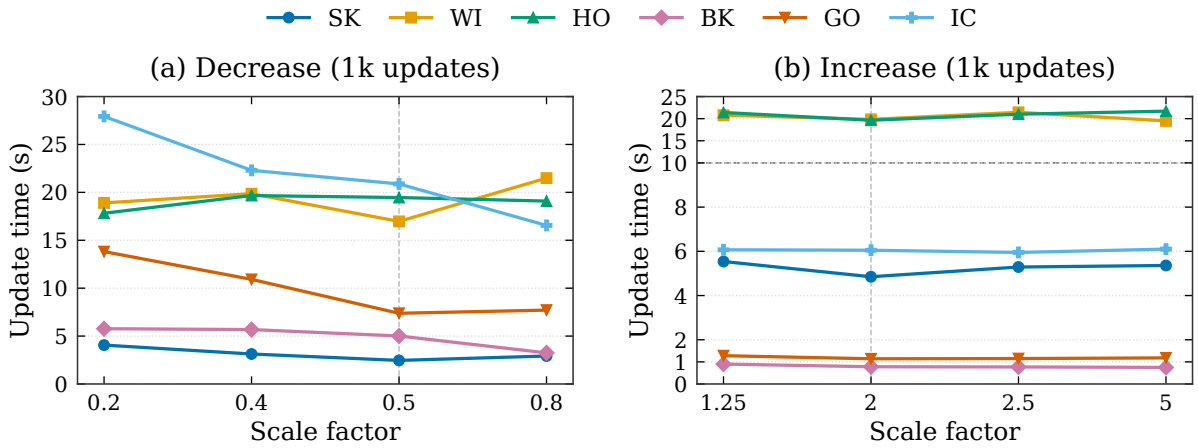


Figure 11: Maintenance of Different Scale Factors